AIMS 5702 Artificial Intelligence in Practice (Fall 2026-27)

AIMS 5702 Assignment 3

Please complete this Colab notebook and:

- Download it as **both** an .ipynb file and a .pdf file (for the .ipynb file, click File -> Download -> Download .ipynb).
- Submit **both** the .pdf and .ipynb files to Blackboard.

Accelerator: Although this training can run on a CPU, a GPU will be much faster, so we recommend using a T4.

Submission deadline: 29th October, 2026 (Thursday)

First, let us import all the libraries needed for this assignment.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import scipy.signal
import math
import time
from typing import List, Union, Dict, Tuple
```

Problem A. Backpropagation

In this problem, let us try to implement backpropagation ourselves.

Requirements:

- You cannot use any PyTorch backward functions.
- You cannot use any automatic differentiation libraries.
- You can use PyTorch forward functions or NumPy utility functions, unless we explicitly state that certain functions cannot be used.

Utility functions for verifying the gradient results

First, let us create a base class from which your classes will inherit. Do not change it.

```
#@title Base class for all your implementations

class CustomizedLayer(object):

  def set_params(self, param_dict: Dict[str, np.ndarray]) -> None:
    """
    Set the parameters of the layer, such as the weight and bias of a linear layer.
```

```
    Args:
      param_dict: A dictionary of parameters.
    """
    raise NotImplementedError

  def forward(self, inputs: List[np.ndarray]) -> np.ndarray:
    """
    Compute the forward pass of the layer.

    Args:
      inputs: A list of input tensors.

    Returns:
      The output of the layer.
    """
    raise NotImplementedError

  def input_gradients(
      self,
      inputs: List[np.ndarray],
      outputs: np.ndarray,
      output_gradient: np.ndarray
  ) -> Union[np.ndarray, List[np.ndarray]]:
    """
    Calculate the input gradients of the layer.
    Args:
      inputs: A list of input tensors.
      outputs: The output of the layer.
      output_gradient: The gradient of the output.
    """
    raise NotImplementedError

  def param_gradients(
      self,
      inputs: List[np.ndarray],
      outputs: np.ndarray,
      output_gradient: np.ndarray
  ) -> Dict[str, np.ndarray]:
    """
    Calculate the parameter gradients of the layer.
    Args:
      inputs: A list of input tensors.
      outputs: The output of the layer.
      output_gradient: The gradient of the output.
    """
    raise NotImplementedError
```

Then, define a few utility functions for testing.

```
#@title Utility functions

def verify_output(
    actual: Union[np.ndarray, List[np.ndarray]],
    reference: Union[np.ndarray, List[np.ndarray]],
    name: Union[str, List[str]],
    atol: float = 1e-6,
    rtol: float = 1e-6
) -> bool:
  """
  Verify whether the actual output is close to the reference output.

  Args:
    actual: The actual output.
    reference: The reference output.
    atol: The absolute tolerance.
```

```
    rtol: The relative tolerance.
  """
  if isinstance(actual, np.ndarray):
    close = (isinstance(reference, np.ndarray)
             and actual.shape == reference.shape
             and np.allclose(actual, reference, atol=atol, rtol=rtol))
    if not close:
      print(f"{name} is not close to the reference output.")
    else:
      print(f"{name} is close to the reference output.")
  elif isinstance(actual, list):
    if not isinstance(reference, list) or len(actual) != len(reference):
      print(f"{name} has a different number of outputs from the reference.")
      return False
    close = all(verify_output(a, b, name[i], atol=atol, rtol=rtol)
                for i, (a, b) in enumerate(zip(actual, reference)))
  else:
    print(f"{name} is not an array or a list of arrays.")
    return False
  return close

def compare_with_actual_layer(
    official_layer: nn.Module,
    customized_layer: CustomizedLayer,
    input_np: List[np.ndarray],
    output_grad_np: np.ndarray,
    parameter_names: List[str],
    param_np: List[np.ndarray],
    a_tol: float = 1e-6,
    r_tol: float = 1e-6,
) -> bool:
  """
  Compare the forward and backward passes of your customized layer with those of the official layer.

  Args:
    official_layer: The official layer.
    customized_layer: The customized layer.
    input_np: The input NumPy arrays.
    output_grad_np: The output gradient NumPy array.
    parameter_names: The names of the parameters.
    param_np: The NumPy arrays of the parameters.
    a_tol: The absolute tolerance.
    r_tol: The relative tolerance.

  Returns:
    True if the output of the customized layer is close to the output of
    the official layer, False otherwise.
  """
  success = True
  input_torch = [
      torch.tensor(x, requires_grad=True) for x in input_np
  ]
  output_grad_torch = torch.tensor(output_grad_np, requires_grad=True)
  for i, name in enumerate(parameter_names):
    setattr(official_layer, name,
            nn.Parameter(torch.tensor(param_np[i], requires_grad=True)))

  # Run the forward and backward passes on the official layer.
  output_official_torch = official_layer(*input_torch)
  output_official_torch.backward(output_grad_torch)
  output_official_np = output_official_torch.detach().numpy()

  # Check output
  output_customized_np = customized_layer.forward(input_np)
  success = verify_output(output_customized_np, output_official_np,
                          "Output", atol=a_tol, rtol=r_tol)
  if not success:
    return success
```

```
  # Check input gradient
  input_grad_customized = customized_layer.input_gradients(
      input_np, output_official_np, output_grad_np)
  input_grad_official = [x.grad.numpy() for x in input_torch]
  success = verify_output(
      input_grad_customized,
      input_grad_official,
      [f'Input Gradient {i}' for i in range(len(input_grad_official))],
      atol=a_tol, rtol=r_tol
  )
  if not success:
    return success

  # Check parameter gradient
  parameter_grad_customized = customized_layer.param_gradients(
      input_np, output_official_np, output_grad_np)
  for name in parameter_names:
    success = verify_output(parameter_grad_customized[name],
                            getattr(official_layer, name).grad.numpy(),
                            f"Parameter Gradient {name}", atol=a_tol, rtol=r_tol)
    if not success:
      return success

  print('All tests passed')
  return success
```

Question A.1 Back-propagation of MUL

Complete the implementation of the class below, then run the test below.

- We have provided the implementation of the forward pass.
- Complete the backward pass in `input_gradients`.
- Note that multiplication does not have parameters, so `param_gradients` returns an empty dictionary.

Hint: For two inputs with the same shape, apply the chain rule to the element-wise product `x * y`.

```
#@title CustomizedMul

class CustomizedMul(CustomizedLayer):
  def __init__(self):
    pass

  def set_params(self, param_dict: Dict[str, np.ndarray]) -> None:
    pass

  def forward(self, inputs: List[np.ndarray]) -> np.ndarray:
    x, y = inputs
    return x * y

  def input_gradients(
    self,
    inputs: List[np.ndarray],
    outputs: np.ndarray,
    output_gradient: np.ndarray
  ) -> Union[np.ndarray, List[np.ndarray]]:
    ######################
    # Your code goes here
    ######################
    x, y = inputs
    # 分别对两个输入应用链式法则
    return [output_gradient * y, output_gradient * x]

  def param_gradients(
```

```
      self,
      inputs: List[np.ndarray],
      outputs: np.ndarray,
      output_gradient: np.ndarray
  ) -> Dict[str, np.ndarray]:
    return {}
```

Run the test below. Do not change it.

```
#@title Run test

def verify_mul():
  for width, height in [(3, 3), (10, 1), (100, 300)]:
    print(f'=== Testing with width={width} and height={height} ===')
    if width == 1:
      x1 = np.random.randn(height)
      x2 = np.random.randn(height)
    else:
      x1 = np.random.randn(height, width)
      x2 = np.random.randn(height, width)
    output_gradient = np.random.randn(*x1.shape)
    customized_mul = CustomizedMul()
    customized_mul.set_params({})
    class TorchMul(nn.Module):
      def __init__(self):
        super(TorchMul, self).__init__()
      def forward(self, x1, x2):
        return x1 * x2
    torch_mul = TorchMul()

    compare_with_actual_layer(
        torch_mul,
        customized_mul,
        [x1, x2],
        output_gradient,
        param_np=[],
        parameter_names=[],
    )

verify_mul()
```

```
=== Testing with width=3 and height=3 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Input Gradient 1 is close to the reference output.
All tests passed
=== Testing with width=10 and height=1 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Input Gradient 1 is close to the reference output.
All tests passed
=== Testing with width=100 and height=300 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Input Gradient 1 is close to the reference output.
All tests passed
```

Question A.2 Back-propagation of ReLU(x * y)

Similarly, complete the implementation of the class below, then run the test below.

- We have provided the implementation of the forward pass.

- Complete the backward pass in `input_gradients`.
- Note that this operation does not have parameters, so `param_gradients` returns an empty dictionary.

Hint: Backpropagate through ReLU first, then through multiplication. For `x * y = 0`, you can assume the gradient of ReLU is 0.

```
#@title CustomizedMulReLU

class CustomizedMulReLU(CustomizedLayer):
  def __init__(self):
    pass

  def set_params(self, param_dict: Dict[str, np.ndarray]) -> None:
    pass

  def forward(self, inputs: List[np.ndarray]) -> np.ndarray:
    x, y = inputs
    return np.maximum(x * y, 0)

  def input_gradients(
    self,
    inputs: List[np.ndarray],
    outputs: np.ndarray,
    output_gradient: np.ndarray
  ) -> Union[np.ndarray, List[np.ndarray]]:
    ######################
    # Your code goes here
    ######################
    x, y = inputs
    # ReLU 只让正数位置的梯度通过
    grad = output_gradient * (x * y > 0)
    return [grad * y, grad * x]

  def param_gradients(
      self,
      inputs: List[np.ndarray],
      outputs: np.ndarray,
      output_gradient: np.ndarray
  ) -> Dict[str, np.ndarray]:
    return {}
```

Run the test below. Do not change it.

```
#@title Run test

def verify_mul_relu():
  for width, height in [(3, 3), (10, 1), (100, 300)]:
    print(f'=== Testing with width={width} and height={height} ===')
    if width == 1:
      x = np.random.randn(height)
      y = np.random.randn(height)
    else:
      x = np.random.randn(height, width)
      y = np.random.randn(height, width)
    output_gradient = np.random.randn(*x.shape)
    customized_mul_relu = CustomizedMulReLU()
    customized_mul_relu.set_params({})
    class TorchMulReLU(nn.Module):
      def forward(self, x, y):
        return torch.relu(x * y)
    torch_mul_relu = TorchMulReLU()

    compare_with_actual_layer(
```

```
        torch_mul_relu,
        customized_mul_relu,
        [x, y],
        output_gradient,
        param_np=[],
        parameter_names=[],
    )

verify_mul_relu()
```

```
=== Testing with width=3 and height=3 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Input Gradient 1 is close to the reference output.
All tests passed
=== Testing with width=10 and height=1 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Input Gradient 1 is close to the reference output.
All tests passed
=== Testing with width=100 and height=300 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Input Gradient 1 is close to the reference output.
All tests passed
```

Question A.3 Back-propagation of Linear + ReLU

Similarly, complete the implementation of the class below, then run the test below.

The forward operation is `ReLU(x @ weight.T + bias)`.

- This time, we also ask you to implement the forward pass.
- Complete the backward pass in `input_gradients` and `param_gradients`.

Hint: Backpropagate through ReLU before the linear layer. You can assume the gradient of ReLU at 0 is 0.

```
#@title CustomizedLinearReLU

class CustomizedLinearReLU(CustomizedLayer):
  def __init__(self, in_features, out_features):
    self.in_features = in_features
    self.out_features = out_features
    self.weight = np.zeros((out_features, in_features))
    self.bias = np.zeros(out_features)

  def set_params(self, param_dict: Dict[str, np.ndarray]):
    self.weight = param_dict['weight']
    self.bias = param_dict['bias']

  def forward(self, inputs: List[np.ndarray]) -> np.ndarray:
    ######################
    # Your code goes here
    ######################
    x = inputs[0]
    return np.maximum(x @ self.weight.T + self.bias, 0)

  def input_gradients(
    self,
    inputs: List[np.ndarray],
    outputs: np.ndarray,
    output_gradient: np.ndarray
  ) -> Union[np.ndarray, List[np.ndarray]]:
    ######################
```

```
    # Your code goes here
    ######################
    grad = output_gradient * (outputs > 0)
    return [grad @ self.weight]

  def param_gradients(
      self,
      inputs: List[np.ndarray],
      outputs: np.ndarray,
      output_gradient: np.ndarray
  ) -> Dict[str, np.ndarray]:
    ######################
    # Your code goes here
    ######################
    x = inputs[0]
    # 先经过 ReLU，再计算线性层的梯度
    grad = output_gradient * (outputs > 0)
    return {'weight': grad.T @ x,
            'bias': grad.sum(axis=0)}
```

Run the test below. Do not change it.

```
#@title Run test

def verify_linear_relu():
  for batch_size, in_features, out_features in [(3, 4, 8), (5, 10, 20), (2, 100, 300)]:
    print(f'=== Testing with in_features={in_features} '
          f'and out_features={out_features} ===')
    input_np = np.random.randn(batch_size, in_features)
    output_gradient = np.random.randn(batch_size, out_features)
    weight = np.random.randn(out_features, in_features)
    bias = np.random.randn(out_features)
    customized_linear_relu = CustomizedLinearReLU(in_features, out_features)
    customized_linear_relu.set_params({'weight': weight, 'bias': bias})
    class TorchLinearReLU(nn.Linear):
      def forward(self, x):
        return torch.relu(super().forward(x))
    torch_linear_relu = TorchLinearReLU(in_features, out_features)

    compare_with_actual_layer(
        torch_linear_relu,
        customized_linear_relu,
        [input_np],
        output_gradient,
        param_np=[weight, bias],
        parameter_names=['weight', 'bias'],
    )

verify_linear_relu()
```

```
=== Testing with in_features=4 and out_features=8 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
=== Testing with in_features=10 and out_features=20 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
=== Testing with in_features=100 and out_features=300 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
```

```
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
```

Question A.4 Back-propagation of Conv

Finally, let us try a convolutional layer. To make your life easier, let us assume:

- Padding is always 1.
- Stride is always 1.
- The kernel is always square with an odd side length.

Similarly, complete the implementation of the class below, then run the test below.

- For your reference, we have provided the forward pass.
- Please complete the backward pass in `input_gradients` and `param_gradients`.

There is an additional requirement for this question:

- You cannot use any functions from NumPy, PyTorch, or other libraries that perform convolution.

Hint: Compute gradients using the zero-padded input, then remove the padded border from the input gradient before returning it.

```
#@title CustomizedConv2d

class CustomizedConv2d(CustomizedLayer):
  def __init__(self, in_channels, out_channels, kernel_size):
    if (kernel_size % 2 == 0):
      raise ValueError(f"Kernel size {kernel_size} must be odd")
    self.in_channels = in_channels
    self.out_channels = out_channels
    self.kernel_size = kernel_size
    self.weight = np.zeros((out_channels, in_channels, kernel_size, kernel_size))
    self.bias = np.zeros(out_channels)

  def set_params(self, param_dict: Dict[str, np.ndarray]):
    self.weight = param_dict['weight']
    self.bias = param_dict['bias']

  def forward(self, inputs: List[np.ndarray]) -> np.ndarray:
    input = inputs[0] # [batch_size, in_channels, height, width]
    batch_size, in_channels, height, width = input.shape
    if self.in_channels != in_channels:
      raise ValueError(f'Input channels {in_channels} do not '
                       f'match expected {self.in_channels}')
    w = self.weight # [out_channels, in_channels, kernel_size, kernel_size]
    b = self.bias
    padded_input = np.pad(input, ((0, 0), (0, 0), (1, 1), (1, 1)),
                          mode='constant')
    out_height = height + 2 - self.kernel_size + 1
    out_width = width + 2 - self.kernel_size + 1
    output = np.zeros((batch_size, self.out_channels, out_height, out_width),
                      np.float64)
    for dx in range(0, self.kernel_size):
      for dy in range(0, self.kernel_size):
        # channel order: [B, out_channels, in_channels, H, W]
        y_start = dy
        y_end = y_start + out_height
        x_start = dx
        x_end = x_start + out_width
        output += (padded_input[:, np.newaxis, :, y_start:y_end, x_start:x_end] *
```

```
                    w[np.newaxis, :, :, np.newaxis, np.newaxis, dy, dx]
        ).sum(axis=2)
    output += b[np.newaxis, :, np.newaxis, np.newaxis]
    return output

  def input_gradients(
    self,
    inputs: List[np.ndarray],
    outputs: np.ndarray,
    output_gradient: np.ndarray
  ) -> Union[np.ndarray, List[np.ndarray]]:
    ######################
    # Your code goes here
    ######################
    x = inputs[0]
    batch_size, in_channels, height, width = x.shape
    out_height, out_width = output_gradient.shape[2:]
    padded_grad = np.zeros((batch_size, in_channels, height + 2, width + 2))

    # 把梯度加回各个核位置对应的输入区域
    for dy in range(self.kernel_size):
      for dx in range(self.kernel_size):
        w = self.weight[:, :, dy, dx]
        grad = (output_gradient[:, :, np.newaxis, :, :] *
                w[np.newaxis, :, :, np.newaxis, np.newaxis]).sum(axis=1)
        padded_grad[:, :, dy:dy + out_height, dx:dx + out_width] += grad

    return [padded_grad[:, :, 1:-1, 1:-1]]

  def param_gradients(
    self,
    inputs: List[np.ndarray],
    outputs: np.ndarray,
    output_gradient: np.ndarray
  ) -> Dict[str, np.ndarray]:
    ######################
    # Your code goes here
    ######################
    x = inputs[0]
    padded_x = np.pad(x, ((0, 0), (0, 0), (1, 1), (1, 1)))
    out_height, out_width = output_gradient.shape[2:]
    weight_grad = np.zeros_like(self.weight)

    # 对所有样本和空间位置累加
    for dy in range(self.kernel_size):
      for dx in range(self.kernel_size):
        patch = padded_x[:, :, dy:dy + out_height, dx:dx + out_width]
        weight_grad[:, :, dy, dx] = (
            output_gradient[:, :, np.newaxis, :, :] *
            patch[:, np.newaxis, :, :, :]
        ).sum(axis=(0, 3, 4))

    bias_grad = output_gradient.sum(axis=(0, 2, 3))
    return {'weight': weight_grad, 'bias': bias_grad}
```

Run the test below. Do not change it.

```
#@title Run test

def verify_conv2d():
  for batch_size, in_channels, out_channels, kernel_size, height, width in [
      (1, 1, 1, 3, 5, 5),
      (1, 4, 5, 3, 11, 11),
      (1, 4, 7, 5, 12, 12),
      (3, 4, 5, 3, 20, 30),
      (1, 5, 7, 5, 256, 256),
```

```
  ]:
    print(f'=== Testing with in_channels={in_channels} '
          f'and out_channels={out_channels} and kernel_size={kernel_size} ===')
    out_height = height + 2 - kernel_size + 1
    out_width = width + 2 - kernel_size + 1
    input_np = np.random.randn(batch_size, in_channels, height, width)
    output_gradient = np.random.randn(batch_size, out_channels, out_height, out_width)
    weight = np.random.randn(out_channels, in_channels, kernel_size, kernel_size)
    bias = np.random.randn(out_channels)
    customized_conv2d = CustomizedConv2d(in_channels, out_channels, kernel_size)
    customized_conv2d.set_params({'weight': weight, 'bias': bias})
    torch_conv2d = nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=1)

    compare_with_actual_layer(
        torch_conv2d,
        customized_conv2d,
        [input_np],
        output_gradient,
        param_np=[weight, bias],
        parameter_names=['weight', 'bias'],
    )

verify_conv2d()
```

```
=== Testing with in_channels=1 and out_channels=1 and kernel_size=3 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
=== Testing with in_channels=4 and out_channels=5 and kernel_size=3 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
=== Testing with in_channels=4 and out_channels=7 and kernel_size=5 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
=== Testing with in_channels=4 and out_channels=5 and kernel_size=3 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
=== Testing with in_channels=5 and out_channels=7 and kernel_size=5 ===
Output is close to the reference output.
Input Gradient 0 is close to the reference output.
Parameter Gradient weight is close to the reference output.
Parameter Gradient bias is close to the reference output.
All tests passed
```

Problem B. ResNet

To simplify network design, we resize the images to 32x32.

```
#@title Load the training dataset

# Define transformations for the training and test sets
```

```
mean_train = 0.2860405743122101
std_train = 0.3530242443084717
transform_train = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((mean_train,), (std_train,))
])

# Apply the same transformation to the training and test sets.
transform_test = transforms.Compose([
    # To simplify network design, we resize the images to 32x32.
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((mean_train,), (std_train,))
])

# Load the FashionMNIST dataset
train_dataset = torchvision.datasets.FashionMNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform_train
)

test_dataset = torchvision.datasets.FashionMNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform_test
)
```

Here are the same utility functions as in Assignment 2. If necessary, feel free to change them.

```
#@title Utility functions for training

def train_one_epoch(
    dataloader: torch.utils.data.DataLoader,
    model: nn.Module,
    loss_fn: nn.Module,
    optimizer: optim.Optimizer,
    device: str='cpu',
    loss_print_iter: int=100
  ):
  num_train_samples = len(dataloader.dataset)

  # Set the model to training mode.
  model.train()
  all_losses = []
  all_acc = []

  for batch_index, (image, label) in enumerate(dataloader):
    image, label = image.to(device), label.to(device)
    pred = model(image)
    loss = loss_fn(pred, label)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    # Calculate the loss
    all_losses.append(loss.item())
    acc = ((pred.argmax(1) == label).type(torch.float).sum().item() /
           image.shape[0])
    all_acc.append(acc)

    if batch_index % loss_print_iter == 0:
      loss, trained_samples = loss.item(), (batch_index + 1) * image.shape[0]
```

```
      print(f'loss: {loss:>7f} '
            f'[{trained_samples:>5d}/{num_train_samples:>5d}] ')

  return all_losses, all_acc

def test_all_samples(
    dataloader: torch.utils.data.DataLoader,
    model: nn.Module,
    loss_fn: nn.Module,
    device: str='cpu'
) -> Tuple[float, float]:
  model.eval()  # Set the model to evaluation mode
  num_testing_samples = len(dataloader.dataset)
  num_batches = len(dataloader)
  test_loss, correct = 0, 0

  # Disable gradient calculation for inference
  with torch.no_grad():
    for image, label in dataloader:
      image, label = image.to(device), label.to(device)
      pred = model(image)
      test_loss += loss_fn(pred, label).item()
      correct += (pred.argmax(1) == label).type(torch.float).sum().item()

  test_loss /= num_batches
  acc = correct / num_testing_samples
  print(f'Test Error: \n Accuracy: {(100*acc):>0.1f}%, '
        f'Avg loss: {test_loss:>8f} \n')
  return test_loss, acc
```

Next, let us select the device.

Using a GPU is recommended.

```
# CUDA 优先，本机 Mac 使用 MPS
if torch.cuda.is_available():
  device = torch.device("cuda")
elif torch.backends.mps.is_available():
  device = torch.device("mps")
else:
  device = torch.device("cpu")
print(f"Using device: {device}")
```

```
Using device: mps
```

In this problem, let us verify why the ResNet structure is important for deep networks.

First, let us create the data loaders. Feel free to change the batch size.

```
# Create data loaders

batch_size = 32
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

Example: A simple CNN.

No coding is required. Just run the following blocks.

```
#@title A utility function for creating the network

def gen_double_conv(in_channels, out_channels):
  return [nn.Conv2d(in_channels, out_channels, 3, padding=1),
          nn.ReLU(),
          nn.Conv2d(out_channels, out_channels, 3, padding=1),
          nn.ReLU()]
```

First, we will define a CNN with 11 convolution/linear layers and try it on FashionMNIST.

```
#@title SimpleCNN

class SimpleCNN(nn.Module):
  def __init__(
    self,
    img_size: int=32,
    in_channels: int=1,
    num_classes: int=10,
  ):
    super().__init__()
    base_channel = 16

    # Starting block. One 2x downsampling operation.
    self.start_conv = nn.Sequential(
        *gen_double_conv(in_channels, base_channel),
    )

    self.start_ds = nn.Sequential(
        nn.AvgPool2d(2),
    )

    # Mid block. No downsampling.
    self.mid_block = nn.Sequential(
        nn.Conv2d(base_channel, base_channel * 2, 3, padding=1),
        nn.ReLU(),
        nn.AvgPool2d(2),
        nn.Conv2d(base_channel * 2, base_channel * 2, 3, padding=1),
        nn.ReLU(),
        # Use ConvTranspose2d to increase the spatial resolution by a factor of 2.
        nn.ConvTranspose2d(base_channel * 2, base_channel, 2, stride=2),
        nn.ReLU(),
    )

    # Last block. Two 2x downsampling operations.
    self.end_conv1 = nn.Sequential(
        *gen_double_conv(base_channel, base_channel * 2)
    )
    self.end_ds1 = nn.Sequential(
        nn.AvgPool2d(2),
    )
    self.end_conv2 = nn.Sequential(
        *gen_double_conv(base_channel * 2, base_channel * 4)
    )

    n_pooling_layer = 3
    last_layer_size = img_size // (2 ** n_pooling_layer)
    self.end_ds2_linear = nn.Sequential(
```

```
        nn.AvgPool2d(2),
        nn.Flatten(),
        nn.Linear(last_layer_size * last_layer_size *
                  base_channel * 4, base_channel * 8),
        nn.ReLU(),
        nn.Linear(base_channel * 8, num_classes)
    )

  def forward(self, x):
    after_start = self.start_ds(self.start_conv(x))
    after_mid = self.mid_block(after_start)
    output = self.end_ds2_linear(
        self.end_conv2(self.end_ds1(self.end_conv1(after_mid))))
    return output

  def get_n_conv_linear_layers(self) -> int:
    all_blocks = [self.start_conv, self.start_ds,
                  self.mid_block,
                  self.end_conv1, self.end_ds1,
                  self.end_conv2, self.end_ds2_linear]
    module_list = (nn.ConvTranspose2d, nn.Conv2d, nn.Linear)
    nlayer = 0
    for block in all_blocks:
      nlayer += len([mod for mod in block.modules()
                     if isinstance(mod, module_list)])
    return nlayer
```

```
#@title Create a model and its optimizer

model = SimpleCNN()
model.to(device)
loss_fn = nn.CrossEntropyLoss()

epochs = 20   # Number of training epochs
optimizer = torch.optim.Adam(
  model.parameters(),    # All trainable parameters.
  lr=1e-4                # Learning rate
)

print(f'The model contains {model.get_n_conv_linear_layers()} conv or '
      f'linear layers')
```

```
The model contains 11 conv or linear layers
```

Run the training code below.

After training, you should be able to achieve 89% accuracy on the test set.

```
#@title Run training

for t in range(epochs):
  print(f"Epoch {t+1}\n-------------------------------")
  start_time =time.time()
  _, _ = train_one_epoch(train_loader, model, loss_fn, optimizer, device=device)
  print(f'One epoch takes {time.time() - start_time}')
  _, _ = test_all_samples(test_loader, model, loss_fn, device=device)

test_all_samples(test_loader, model, loss_fn, device=device)

print("Training done!")
```

```
Epoch 1
-------------------------------
loss: 2.292041 [   32/60000]
```

```
loss: 2.298976 [ 3232/60000]
loss: 1.361628 [ 6432/60000]
loss: 1.128845 [ 9632/60000]
loss: 0.803277 [12832/60000]
loss: 0.927519 [16032/60000]
loss: 0.797342 [19232/60000]
loss: 0.652546 [22432/60000]
loss: 0.827107 [25632/60000]
loss: 0.554794 [28832/60000]
loss: 0.670301 [32032/60000]
loss: 0.470756 [35232/60000]
loss: 0.732368 [38432/60000]
loss: 0.565886 [41632/60000]
loss: 0.629744 [44832/60000]
loss: 0.624358 [48032/60000]
loss: 0.738034 [51232/60000]
loss: 0.525354 [54432/60000]
loss: 0.701658 [57632/60000]
One epoch takes 24.036652088165283
Test Error:
 Accuracy: 76.3%, Avg loss: 0.618299

Epoch 2
-------------------------------
loss: 0.365552 [   32/60000]
loss: 0.665926 [ 3232/60000]
loss: 0.682513 [ 6432/60000]
loss: 0.564250 [ 9632/60000]
loss: 0.284552 [12832/60000]
loss: 0.486772 [16032/60000]
loss: 0.518930 [19232/60000]
loss: 0.553954 [22432/60000]
loss: 0.411968 [25632/60000]
loss: 0.861329 [28832/60000]
loss: 0.321397 [32032/60000]
loss: 0.413855 [35232/60000]
loss: 0.355981 [38432/60000]
loss: 0.615475 [41632/60000]
loss: 0.587763 [44832/60000]
loss: 0.532371 [48032/60000]
loss: 0.368934 [51232/60000]
loss: 0.653315 [54432/60000]
loss: 0.327981 [57632/60000]
One epoch takes 22.31171488761902
Test Error:
 Accuracy: 80.0%, Avg loss: 0.531504

Epoch 3
-------------------------------
loss: 0.485861 [   32/60000]
loss: 0.429709 [ 3232/60000]
loss: 0.624170 [ 6432/60000]
loss: 0.385918 [ 9632/60000]
loss: 0.381280 [12832/60000]
loss: 0.527503 [16032/60000]
loss: 0.345532 [19232/60000]
loss: 0.410127 [22432/60000]
loss: 0.498007 [25632/60000]
loss: 0.751658 [28832/60000]
loss: 0.528492 [32032/60000]
loss: 0.621898 [35232/60000]
loss: 0.290304 [38432/60000]
loss: 0.746376 [41632/60000]
loss: 0.590344 [44832/60000]
loss: 0.824296 [48032/60000]
loss: 0.562198 [51232/60000]
loss: 0.349743 [54432/60000]
loss: 0.610860 [57632/60000]
One epoch takes 22.499632120132446
```

```
Test Error:
 Accuracy: 81.5%, Avg loss: 0.508384

Epoch 4
-------------------------------
loss: 0.366729 [   32/60000]
loss: 0.483393 [ 3232/60000]
loss: 0.321570 [ 6432/60000]
loss: 0.583683 [ 9632/60000]
loss: 0.528524 [12832/60000]
loss: 0.520819 [16032/60000]
loss: 0.405058 [19232/60000]
loss: 0.365987 [22432/60000]
loss: 0.416381 [25632/60000]
loss: 0.410007 [28832/60000]
loss: 0.873549 [32032/60000]
loss: 0.549532 [35232/60000]
loss: 0.634455 [38432/60000]
loss: 0.271851 [41632/60000]
loss: 0.359647 [44832/60000]
loss: 0.338757 [48032/60000]
loss: 0.336999 [51232/60000]
loss: 0.448662 [54432/60000]
loss: 0.352574 [57632/60000]
One epoch takes 23.215938091278076
Test Error:
 Accuracy: 83.0%, Avg loss: 0.444645

Epoch 5
-------------------------------
loss: 0.352772 [   32/60000]
loss: 0.271295 [ 3232/60000]
loss: 0.250642 [ 6432/60000]
loss: 0.369091 [ 9632/60000]
loss: 0.885205 [12832/60000]
loss: 0.323968 [16032/60000]
loss: 0.258833 [19232/60000]
loss: 0.248023 [22432/60000]
loss: 0.471563 [25632/60000]
loss: 0.695065 [28832/60000]
loss: 0.347223 [32032/60000]
loss: 0.642648 [35232/60000]
loss: 0.663097 [38432/60000]
loss: 0.411019 [41632/60000]
loss: 0.402683 [44832/60000]
loss: 0.490232 [48032/60000]
loss: 0.255042 [51232/60000]
loss: 0.403579 [54432/60000]
loss: 0.492072 [57632/60000]
One epoch takes 22.830431938171387
Test Error:
 Accuracy: 83.1%, Avg loss: 0.438126

Epoch 6
-------------------------------
loss: 0.387312 [   32/60000]
loss: 0.492687 [ 3232/60000]
loss: 0.278835 [ 6432/60000]
loss: 0.297881 [ 9632/60000]
loss: 0.444644 [12832/60000]
loss: 0.455286 [16032/60000]
loss: 0.318384 [19232/60000]
loss: 0.432472 [22432/60000]
loss: 0.465933 [25632/60000]
loss: 0.306787 [28832/60000]
loss: 0.471593 [32032/60000]
loss: 0.486125 [35232/60000]
loss: 0.323341 [38432/60000]
loss: 0.196864 [41632/60000]
```

```
loss: 0.326911 [44832/60000]
loss: 0.366606 [48032/60000]
loss: 0.371078 [51232/60000]
loss: 0.315255 [54432/60000]
loss: 0.468234 [57632/60000]
One epoch takes 22.04094886779785
Test Error:
 Accuracy: 85.2%, Avg loss: 0.391729

Epoch 7
-------------------------------
loss: 0.201008 [   32/60000]
loss: 0.250930 [ 3232/60000]
loss: 0.519767 [ 6432/60000]
loss: 0.418961 [ 9632/60000]
loss: 0.461886 [12832/60000]
loss: 0.285600 [16032/60000]
loss: 0.398937 [19232/60000]
loss: 0.385182 [22432/60000]
loss: 0.457919 [25632/60000]
loss: 0.315240 [28832/60000]
loss: 0.482134 [32032/60000]
loss: 0.270558 [35232/60000]
loss: 0.310400 [38432/60000]
loss: 0.586990 [41632/60000]
loss: 0.403946 [44832/60000]
loss: 0.259204 [48032/60000]
loss: 0.433572 [51232/60000]
loss: 0.343644 [54432/60000]
loss: 0.250565 [57632/60000]
One epoch takes 21.88052797317505
Test Error:
 Accuracy: 86.3%, Avg loss: 0.366766

Epoch 8
-------------------------------
loss: 0.337046 [   32/60000]
loss: 0.316689 [ 3232/60000]
loss: 0.419115 [ 6432/60000]
loss: 0.295915 [ 9632/60000]
loss: 0.222337 [12832/60000]
loss: 0.575131 [16032/60000]
loss: 0.271591 [19232/60000]
loss: 0.655651 [22432/60000]
loss: 0.414161 [25632/60000]
loss: 0.344748 [28832/60000]
loss: 0.449341 [32032/60000]
loss: 0.558762 [35232/60000]
loss: 0.277534 [38432/60000]
loss: 0.230568 [41632/60000]
loss: 0.176030 [44832/60000]
loss: 0.352011 [48032/60000]
loss: 0.425804 [51232/60000]
loss: 0.427469 [54432/60000]
loss: 0.282644 [57632/60000]
One epoch takes 23.62432599067688
Test Error:
 Accuracy: 86.7%, Avg loss: 0.354497

Epoch 9
-------------------------------
loss: 0.415342 [   32/60000]
loss: 0.724396 [ 3232/60000]
loss: 0.337896 [ 6432/60000]
loss: 0.395464 [ 9632/60000]
loss: 0.298412 [12832/60000]
loss: 0.225127 [16032/60000]
loss: 0.238509 [19232/60000]
loss: 0.342778 [22432/60000]
```

```
loss: 0.345271 [25632/60000]
loss: 0.225513 [28832/60000]
loss: 0.238903 [32032/60000]
loss: 0.385777 [35232/60000]
loss: 0.178223 [38432/60000]
loss: 0.449507 [41632/60000]
loss: 0.317579 [44832/60000]
loss: 0.387945 [48032/60000]
loss: 0.337970 [51232/60000]
loss: 0.200394 [54432/60000]
loss: 0.431278 [57632/60000]
One epoch takes 24.572652101516724
Test Error:
 Accuracy: 87.4%, Avg loss: 0.340344

Epoch 10
-------------------------------
loss: 0.512283 [   32/60000]
loss: 0.250130 [ 3232/60000]
loss: 0.300683 [ 6432/60000]
loss: 0.515811 [ 9632/60000]
loss: 0.288460 [12832/60000]
loss: 0.295511 [16032/60000]
loss: 0.460448 [19232/60000]
loss: 0.235704 [22432/60000]
loss: 0.140926 [25632/60000]
loss: 0.245122 [28832/60000]
loss: 0.321791 [32032/60000]
loss: 0.186431 [35232/60000]
loss: 0.589640 [38432/60000]
loss: 0.231881 [41632/60000]
loss: 0.373242 [44832/60000]
loss: 0.192182 [48032/60000]
loss: 0.270939 [51232/60000]
loss: 0.324950 [54432/60000]
loss: 0.157043 [57632/60000]
One epoch takes 22.856987237930298
Test Error:
 Accuracy: 88.0%, Avg loss: 0.326464

Epoch 11
-------------------------------
loss: 0.382261 [   32/60000]
loss: 0.393432 [ 3232/60000]
loss: 0.168008 [ 6432/60000]
loss: 0.604268 [ 9632/60000]
loss: 0.259647 [12832/60000]
loss: 0.172717 [16032/60000]
loss: 0.321354 [19232/60000]
loss: 0.206773 [22432/60000]
loss: 0.216296 [25632/60000]
loss: 0.167318 [28832/60000]
loss: 0.171915 [32032/60000]
loss: 0.378523 [35232/60000]
loss: 0.231048 [38432/60000]
loss: 0.192295 [41632/60000]
loss: 0.361485 [44832/60000]
loss: 0.241395 [48032/60000]
loss: 0.277404 [51232/60000]
loss: 0.199569 [54432/60000]
loss: 0.301997 [57632/60000]
One epoch takes 22.41439986228943
Test Error:
 Accuracy: 88.3%, Avg loss: 0.318298

Epoch 12
-------------------------------
loss: 0.174916 [   32/60000]
loss: 0.419794 [ 3232/60000]
```

```
loss: 0.242948 [ 6432/60000]
loss: 0.076191 [ 9632/60000]
loss: 0.234892 [12832/60000]
loss: 0.247172 [16032/60000]
loss: 0.166400 [19232/60000]
loss: 0.306068 [22432/60000]
loss: 0.286797 [25632/60000]
loss: 0.201571 [28832/60000]
loss: 0.244897 [32032/60000]
loss: 0.468784 [35232/60000]
loss: 0.113132 [38432/60000]
loss: 0.440370 [41632/60000]
loss: 0.165821 [44832/60000]
loss: 0.580822 [48032/60000]
loss: 0.126828 [51232/60000]
loss: 0.229946 [54432/60000]
loss: 0.254097 [57632/60000]
One epoch takes 22.381027936935425
Test Error:
 Accuracy: 88.4%, Avg loss: 0.311865

Epoch 13
-------------------------------
loss: 0.471547 [   32/60000]
loss: 0.096326 [ 3232/60000]
loss: 0.129998 [ 6432/60000]
loss: 0.387897 [ 9632/60000]
loss: 0.116988 [12832/60000]
loss: 0.196676 [16032/60000]
loss: 0.247902 [19232/60000]
loss: 0.471971 [22432/60000]
loss: 0.280643 [25632/60000]
loss: 0.083571 [28832/60000]
loss: 0.504729 [32032/60000]
loss: 0.220506 [35232/60000]
loss: 0.248336 [38432/60000]
loss: 0.282677 [41632/60000]
loss: 0.255283 [44832/60000]
loss: 0.379213 [48032/60000]
loss: 0.080558 [51232/60000]
loss: 0.174408 [54432/60000]
loss: 0.195963 [57632/60000]
One epoch takes 22.974218130111694
Test Error:
 Accuracy: 89.1%, Avg loss: 0.297463

Epoch 14
-------------------------------
loss: 0.174442 [   32/60000]
loss: 0.223958 [ 3232/60000]
loss: 0.222532 [ 6432/60000]
loss: 0.218337 [ 9632/60000]
loss: 0.387932 [12832/60000]
loss: 0.578482 [16032/60000]
loss: 0.195138 [19232/60000]
loss: 0.136858 [22432/60000]
loss: 0.177705 [25632/60000]
loss: 0.125919 [28832/60000]
loss: 0.159355 [32032/60000]
loss: 0.277414 [35232/60000]
loss: 0.150935 [38432/60000]
loss: 0.138154 [41632/60000]
loss: 0.308551 [44832/60000]
loss: 0.075330 [48032/60000]
loss: 0.251997 [51232/60000]
loss: 0.184997 [54432/60000]
loss: 0.204251 [57632/60000]
One epoch takes 23.31810712814331
Test Error:
```

```
 Accuracy: 89.4%, Avg loss: 0.287086

Epoch 15
-------------------------------
loss: 0.338274 [   32/60000]
loss: 0.400265 [ 3232/60000]
loss: 0.247385 [ 6432/60000]
loss: 0.158993 [ 9632/60000]
loss: 0.106554 [12832/60000]
loss: 0.156673 [16032/60000]
loss: 0.205101 [19232/60000]
loss: 0.165919 [22432/60000]
loss: 0.216718 [25632/60000]
loss: 0.322033 [28832/60000]
loss: 0.161497 [32032/60000]
loss: 0.259608 [35232/60000]
loss: 0.168157 [38432/60000]
loss: 0.248775 [41632/60000]
loss: 0.296995 [44832/60000]
loss: 0.492167 [48032/60000]
loss: 0.350727 [51232/60000]
loss: 0.136146 [54432/60000]
loss: 0.183226 [57632/60000]
One epoch takes 22.805256843566895
Test Error:
 Accuracy: 89.5%, Avg loss: 0.282589

Epoch 16
-------------------------------
loss: 0.351815 [   32/60000]
loss: 0.183706 [ 3232/60000]
loss: 0.264765 [ 6432/60000]
loss: 0.280729 [ 9632/60000]
loss: 0.209995 [12832/60000]
loss: 0.350460 [16032/60000]
loss: 0.235897 [19232/60000]
loss: 0.187873 [22432/60000]
loss: 0.142915 [25632/60000]
loss: 0.191867 [28832/60000]
loss: 0.190035 [32032/60000]
loss: 0.308808 [35232/60000]
loss: 0.167746 [38432/60000]
loss: 0.477416 [41632/60000]
loss: 0.300994 [44832/60000]
loss: 0.206224 [48032/60000]
loss: 0.297500 [51232/60000]
loss: 0.060684 [54432/60000]
loss: 0.228133 [57632/60000]
One epoch takes 23.07361888885498
Test Error:
 Accuracy: 89.9%, Avg loss: 0.277844

Epoch 17
-------------------------------
loss: 0.214422 [   32/60000]
loss: 0.103383 [ 3232/60000]
loss: 0.119776 [ 6432/60000]
loss: 0.800640 [ 9632/60000]
loss: 0.246208 [12832/60000]
loss: 0.312117 [16032/60000]
loss: 0.195671 [19232/60000]
loss: 0.167172 [22432/60000]
loss: 0.339916 [25632/60000]
loss: 0.198981 [28832/60000]
loss: 0.143949 [32032/60000]
loss: 0.207974 [35232/60000]
loss: 0.505420 [38432/60000]
loss: 0.286926 [41632/60000]
loss: 0.288937 [44832/60000]
```

```
loss: 0.351936 [48032/60000]
loss: 0.233854 [51232/60000]
loss: 0.308425 [54432/60000]
loss: 0.184137 [57632/60000]
One epoch takes 22.895431756973267
Test Error:
 Accuracy: 89.8%, Avg loss: 0.284917

Epoch 18
-------------------------------
loss: 0.104985 [   32/60000]
loss: 0.106810 [ 3232/60000]
loss: 0.267231 [ 6432/60000]
loss: 0.236852 [ 9632/60000]
loss: 0.318731 [12832/60000]
loss: 0.355644 [16032/60000]
loss: 0.390273 [19232/60000]
loss: 0.161672 [22432/60000]
loss: 0.080296 [25632/60000]
loss: 0.245235 [28832/60000]
loss: 0.242151 [32032/60000]
loss: 0.431237 [35232/60000]
loss: 0.201718 [38432/60000]
loss: 0.296536 [41632/60000]
loss: 0.179949 [44832/60000]
loss: 0.335528 [48032/60000]
loss: 0.145146 [51232/60000]
loss: 0.313657 [54432/60000]
loss: 0.162044 [57632/60000]
One epoch takes 22.57425880432129
Test Error:
 Accuracy: 89.9%, Avg loss: 0.287070

Epoch 19
-------------------------------
loss: 0.195350 [   32/60000]
loss: 0.120875 [ 3232/60000]
loss: 0.154862 [ 6432/60000]
loss: 0.147108 [ 9632/60000]
loss: 0.078232 [12832/60000]
loss: 0.118442 [16032/60000]
loss: 0.311735 [19232/60000]
loss: 0.133669 [22432/60000]
loss: 0.200809 [25632/60000]
loss: 0.117661 [28832/60000]
loss: 0.153355 [32032/60000]
loss: 0.042800 [35232/60000]
loss: 0.231983 [38432/60000]
loss: 0.257771 [41632/60000]
loss: 0.114836 [44832/60000]
loss: 0.232407 [48032/60000]
loss: 0.400359 [51232/60000]
loss: 0.092168 [54432/60000]
loss: 0.136033 [57632/60000]
One epoch takes 23.138389110565186
Test Error:
 Accuracy: 89.8%, Avg loss: 0.286725

Epoch 20
-------------------------------
loss: 0.052446 [   32/60000]
loss: 0.262618 [ 3232/60000]
loss: 0.311189 [ 6432/60000]
loss: 0.158410 [ 9632/60000]
loss: 0.144609 [12832/60000]
loss: 0.301418 [16032/60000]
loss: 0.306708 [19232/60000]
loss: 0.274904 [22432/60000]
loss: 0.086119 [25632/60000]
```

```
loss: 0.113498 [28832/60000]
loss: 0.062043 [32032/60000]
loss: 0.128113 [35232/60000]
loss: 0.190949 [38432/60000]
loss: 0.133664 [41632/60000]
loss: 0.112950 [44832/60000]
loss: 0.261470 [48032/60000]
loss: 0.152628 [51232/60000]
loss: 0.110361 [54432/60000]
loss: 0.332877 [57632/60000]
One epoch takes 24.931922912597656
Test Error:
 Accuracy: 90.8%, Avg loss: 0.257335

Test Error:
 Accuracy: 90.8%, Avg loss: 0.257335

Training done!
```

Problem B.1. Deeper CNN

Now, please change the network we have provided by making multiple independent copies of `self.mid_block`. Keep the remaining modules, their names and channel settings unchanged. The network you have created should have at least 30 conv/linear layers.

```python
class DeeperCNN(nn.Module):
  def __init__(
    self,
    img_size: int=32,
    in_channels: int=1,
    num_classes: int=10,
  ):
    ######################
    # Your code goes here
    ######################
    super().__init__()
    base_channel = 16

    self.start_conv = nn.Sequential(
        *gen_double_conv(in_channels, base_channel),
    )
    self.start_ds = nn.Sequential(nn.AvgPool2d(2))

    # 每个块都重新创建，不共享权重
    self.mid_block = nn.ModuleList([
        nn.Sequential(
            nn.Conv2d(base_channel, base_channel * 2, 3, padding=1),
            nn.ReLU(),
            nn.AvgPool2d(2),
            nn.Conv2d(base_channel * 2, base_channel * 2, 3, padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(base_channel * 2, base_channel, 2, stride=2),
            nn.ReLU(),
        ) for _ in range(8)
    ])

    self.end_conv1 = nn.Sequential(
        *gen_double_conv(base_channel, base_channel * 2)
    )
    self.end_ds1 = nn.Sequential(nn.AvgPool2d(2))
    self.end_conv2 = nn.Sequential(
        *gen_double_conv(base_channel * 2, base_channel * 4)
    )
```

```
    n_pooling_layer = 3
    last_layer_size = img_size // (2 ** n_pooling_layer)
    self.end_ds2_linear = nn.Sequential(
        nn.AvgPool2d(2),
        nn.Flatten(),
        nn.Linear(last_layer_size * last_layer_size *
                  base_channel * 4, base_channel * 8),
        nn.ReLU(),
        nn.Linear(base_channel * 8, num_classes)
    )

  def forward(self, x):
    ######################
    # Your code goes here
    ######################
    after_start = self.start_ds(self.start_conv(x))
    after_mid = after_start
    for block in self.mid_block:
      after_mid = block(after_mid)
    output = self.end_ds2_linear(
        self.end_conv2(self.end_ds1(self.end_conv1(after_mid))))
    return output

  def get_n_conv_linear_layers(self) -> int:
    ######################
    # Your code goes here
    ######################
    return sum(isinstance(mod, (nn.ConvTranspose2d, nn.Conv2d, nn.Linear))
               for mod in self.modules())
```

```
#@title Create a model and its optimizer

model = DeeperCNN()
model.to(device)
loss_fn = nn.CrossEntropyLoss()

epochs = 20   # Number of training epochs
optimizer = torch.optim.Adam(
  model.parameters(),    # All trainable parameters.
  lr=1e-4                # Learning rate
)

print(f'The model contains {model.get_n_conv_linear_layers()} conv or '
      f'linear layers')
```

```
The model contains 32 conv or linear layers
```

Run the training code below.

Interestingly, you will find that with a deeper network, the accuracy actually goes down.

```
#@title Run training

for t in range(epochs):
  print(f"Epoch {t+1}\n-------------------------------")
  start_time =time.time()
  _, _ = train_one_epoch(train_loader, model, loss_fn, optimizer, device=device)
  print(f'One epoch takes {time.time() - start_time}')
  _, _ = test_all_samples(test_loader, model, loss_fn, device=device)

test_all_samples(test_loader, model, loss_fn, device=device)

print("Training done!")
```

```
Epoch 1
-------------------------------
loss: 2.318420 [   32/60000]
loss: 2.283867 [ 3232/60000]
loss: 2.311431 [ 6432/60000]
loss: 2.315685 [ 9632/60000]
loss: 2.304106 [12832/60000]
loss: 2.308261 [16032/60000]
loss: 2.304962 [19232/60000]
loss: 2.296997 [22432/60000]
loss: 2.312918 [25632/60000]
loss: 2.302586 [28832/60000]
loss: 2.306840 [32032/60000]
loss: 2.304224 [35232/60000]
loss: 2.301081 [38432/60000]
loss: 2.300606 [41632/60000]
loss: 2.302626 [44832/60000]
loss: 2.306006 [48032/60000]
loss: 2.301953 [51232/60000]
loss: 2.305034 [54432/60000]
loss: 2.305147 [57632/60000]
One epoch takes 42.4262580871582
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302664

Epoch 2
-------------------------------
loss: 2.304302 [   32/60000]
loss: 2.303645 [ 3232/60000]
loss: 2.302605 [ 6432/60000]
loss: 2.301414 [ 9632/60000]
loss: 2.300265 [12832/60000]
loss: 2.307364 [16032/60000]
loss: 2.301617 [19232/60000]
loss: 2.301803 [22432/60000]
loss: 2.305246 [25632/60000]
loss: 2.303670 [28832/60000]
loss: 2.304075 [32032/60000]
loss: 2.300476 [35232/60000]
loss: 2.301338 [38432/60000]
loss: 2.304681 [41632/60000]
loss: 2.304433 [44832/60000]
loss: 2.300367 [48032/60000]
loss: 2.303165 [51232/60000]
loss: 2.297485 [54432/60000]
loss: 2.303000 [57632/60000]
One epoch takes 44.7683470249176
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302756

Epoch 3
-------------------------------
loss: 2.301039 [   32/60000]
loss: 2.304804 [ 3232/60000]
loss: 2.303081 [ 6432/60000]
loss: 2.304622 [ 9632/60000]
loss: 2.304093 [12832/60000]
loss: 2.301781 [16032/60000]
loss: 2.305322 [19232/60000]
loss: 2.303782 [22432/60000]
loss: 2.304773 [25632/60000]
loss: 2.302497 [28832/60000]
loss: 2.302523 [32032/60000]
loss: 2.307858 [35232/60000]
loss: 2.305486 [38432/60000]
loss: 2.305680 [41632/60000]
loss: 2.301345 [44832/60000]
loss: 2.303274 [48032/60000]
```

```
loss: 2.305600 [51232/60000]
loss: 2.301503 [54432/60000]
loss: 2.305480 [57632/60000]
One epoch takes 43.11914610862732
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302607

Epoch 4
-------------------------------
loss: 2.302958 [   32/60000]
loss: 2.301735 [ 3232/60000]
loss: 2.305855 [ 6432/60000]
loss: 2.305261 [ 9632/60000]
loss: 2.306762 [12832/60000]
loss: 2.301491 [16032/60000]
loss: 2.301239 [19232/60000]
loss: 2.300278 [22432/60000]
loss: 2.301547 [25632/60000]
loss: 2.300164 [28832/60000]
loss: 2.306339 [32032/60000]
loss: 2.301463 [35232/60000]
loss: 2.299533 [38432/60000]
loss: 2.306138 [41632/60000]
loss: 2.299824 [44832/60000]
loss: 2.303568 [48032/60000]
loss: 2.302270 [51232/60000]
loss: 2.302553 [54432/60000]
loss: 2.302524 [57632/60000]
One epoch takes 42.19586682319641
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302616

Epoch 5
-------------------------------
loss: 2.298023 [   32/60000]
loss: 2.302785 [ 3232/60000]
loss: 2.302669 [ 6432/60000]
loss: 2.302565 [ 9632/60000]
loss: 2.301918 [12832/60000]
loss: 2.299112 [16032/60000]
loss: 2.304672 [19232/60000]
loss: 2.303096 [22432/60000]
loss: 2.303526 [25632/60000]
loss: 2.304188 [28832/60000]
loss: 2.302660 [32032/60000]
loss: 2.302978 [35232/60000]
loss: 2.300850 [38432/60000]
loss: 2.302741 [41632/60000]
loss: 2.301769 [44832/60000]
loss: 2.305807 [48032/60000]
loss: 2.300751 [51232/60000]
loss: 2.304074 [54432/60000]
loss: 2.301351 [57632/60000]
One epoch takes 53.16389298439026
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302725

Epoch 6
-------------------------------
loss: 2.305628 [   32/60000]
loss: 2.302865 [ 3232/60000]
loss: 2.302852 [ 6432/60000]
loss: 2.303230 [ 9632/60000]
loss: 2.302620 [12832/60000]
loss: 2.302411 [16032/60000]
loss: 2.301605 [19232/60000]
loss: 2.302059 [22432/60000]
loss: 2.301278 [25632/60000]
loss: 2.300349 [28832/60000]
```

```
loss: 2.304608 [32032/60000]
loss: 2.302808 [35232/60000]
loss: 2.303211 [38432/60000]
loss: 2.300234 [41632/60000]
loss: 2.304111 [44832/60000]
loss: 2.302988 [48032/60000]
loss: 2.302416 [51232/60000]
loss: 2.301488 [54432/60000]
loss: 2.301153 [57632/60000]
One epoch takes 60.02863311767578
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302652

Epoch 7
-------------------------------
loss: 2.303767 [   32/60000]
loss: 2.306110 [ 3232/60000]
loss: 2.302331 [ 6432/60000]
loss: 2.303288 [ 9632/60000]
loss: 2.303729 [12832/60000]
loss: 2.302563 [16032/60000]
loss: 2.304999 [19232/60000]
loss: 2.301324 [22432/60000]
loss: 2.302204 [25632/60000]
loss: 2.302299 [28832/60000]
loss: 2.302138 [32032/60000]
loss: 2.302728 [35232/60000]
loss: 2.301733 [38432/60000]
loss: 2.300858 [41632/60000]
loss: 2.303092 [44832/60000]
loss: 2.302868 [48032/60000]
loss: 2.303091 [51232/60000]
loss: 2.299773 [54432/60000]
loss: 2.303152 [57632/60000]
One epoch takes 45.96259093284607
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302633

Epoch 8
-------------------------------
loss: 2.304516 [   32/60000]
loss: 2.302369 [ 3232/60000]
loss: 2.302841 [ 6432/60000]
loss: 2.301484 [ 9632/60000]
loss: 2.299983 [12832/60000]
loss: 2.301948 [16032/60000]
loss: 2.302794 [19232/60000]
loss: 2.302300 [22432/60000]
loss: 2.305046 [25632/60000]
loss: 2.301877 [28832/60000]
loss: 2.302273 [32032/60000]
loss: 2.305382 [35232/60000]
loss: 2.301753 [38432/60000]
loss: 2.303608 [41632/60000]
loss: 2.303054 [44832/60000]
loss: 2.304072 [48032/60000]
loss: 2.303343 [51232/60000]
loss: 2.299579 [54432/60000]
loss: 2.301813 [57632/60000]
One epoch takes 50.41440486907959
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302621

Epoch 9
-------------------------------
loss: 2.302898 [   32/60000]
loss: 2.303048 [ 3232/60000]
loss: 2.301702 [ 6432/60000]
loss: 2.303058 [ 9632/60000]
```

```
loss: 2.303087 [12832/60000]
loss: 2.304622 [16032/60000]
loss: 2.302926 [19232/60000]
loss: 2.302474 [22432/60000]
loss: 2.302173 [25632/60000]
loss: 2.301121 [28832/60000]
loss: 2.299759 [32032/60000]
loss: 2.301507 [35232/60000]
loss: 2.301828 [38432/60000]
loss: 2.303948 [41632/60000]
loss: 2.303672 [44832/60000]
loss: 2.303846 [48032/60000]
loss: 2.301846 [51232/60000]
loss: 2.301270 [54432/60000]
loss: 2.303915 [57632/60000]
One epoch takes 43.842609167099
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302597

Epoch 10
-------------------------------
loss: 2.304178 [   32/60000]
loss: 2.303205 [ 3232/60000]
loss: 2.299943 [ 6432/60000]
loss: 2.302001 [ 9632/60000]
loss: 2.303064 [12832/60000]
loss: 2.301754 [16032/60000]
loss: 2.303492 [19232/60000]
loss: 2.302242 [22432/60000]
loss: 2.304431 [25632/60000]
loss: 2.302251 [28832/60000]
loss: 2.304291 [32032/60000]
loss: 2.303341 [35232/60000]
loss: 2.301216 [38432/60000]
loss: 2.302273 [41632/60000]
loss: 2.301612 [44832/60000]
loss: 2.302133 [48032/60000]
loss: 2.302822 [51232/60000]
loss: 2.303242 [54432/60000]
loss: 2.301974 [57632/60000]
One epoch takes 41.61956286430359
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302597

Epoch 11
-------------------------------
loss: 2.301611 [   32/60000]
loss: 2.304980 [ 3232/60000]
loss: 2.304233 [ 6432/60000]
loss: 2.303915 [ 9632/60000]
loss: 2.301376 [12832/60000]
loss: 2.303661 [16032/60000]
loss: 2.302045 [19232/60000]
loss: 2.302660 [22432/60000]
loss: 2.301178 [25632/60000]
loss: 2.305813 [28832/60000]
loss: 2.299733 [32032/60000]
loss: 2.302533 [35232/60000]
loss: 2.300674 [38432/60000]
loss: 2.300426 [41632/60000]
loss: 2.304085 [44832/60000]
loss: 2.301135 [48032/60000]
loss: 2.303478 [51232/60000]
loss: 2.305065 [54432/60000]
loss: 2.300414 [57632/60000]
One epoch takes 45.912606954574585
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302624
```

```
Epoch 12
-------------------------------
loss: 2.304509 [   32/60000]
loss: 2.304569 [ 3232/60000]
loss: 2.301113 [ 6432/60000]
loss: 2.303008 [ 9632/60000]
loss: 2.302145 [12832/60000]
loss: 2.299962 [16032/60000]
loss: 2.301472 [19232/60000]
loss: 2.302653 [22432/60000]
loss: 2.302878 [25632/60000]
loss: 2.304303 [28832/60000]
loss: 2.302615 [32032/60000]
loss: 2.303124 [35232/60000]
loss: 2.303204 [38432/60000]
loss: 2.302689 [41632/60000]
loss: 2.301134 [44832/60000]
loss: 2.301933 [48032/60000]
loss: 2.303157 [51232/60000]
loss: 2.303229 [54432/60000]
loss: 2.302036 [57632/60000]
One epoch takes 46.4138879776001
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302697

Epoch 13
-------------------------------
loss: 2.299765 [   32/60000]
loss: 2.304137 [ 3232/60000]
loss: 2.302783 [ 6432/60000]
loss: 2.300975 [ 9632/60000]
loss: 2.300792 [12832/60000]
loss: 2.302721 [16032/60000]
loss: 2.302470 [19232/60000]
loss: 2.299962 [22432/60000]
loss: 2.304356 [25632/60000]
loss: 2.303892 [28832/60000]
loss: 2.303502 [32032/60000]
loss: 2.302620 [35232/60000]
loss: 2.302592 [38432/60000]
loss: 2.302064 [41632/60000]
loss: 2.303226 [44832/60000]
loss: 2.303918 [48032/60000]
loss: 2.303718 [51232/60000]
loss: 2.301543 [54432/60000]
loss: 2.302567 [57632/60000]
One epoch takes 50.833677768707275
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302593

Epoch 14
-------------------------------
loss: 2.303323 [   32/60000]
loss: 2.302108 [ 3232/60000]
loss: 2.302797 [ 6432/60000]
loss: 2.302427 [ 9632/60000]
loss: 2.301763 [12832/60000]
loss: 2.298857 [16032/60000]
loss: 2.301872 [19232/60000]
loss: 2.302220 [22432/60000]
loss: 2.299395 [25632/60000]
loss: 2.299324 [28832/60000]
loss: 2.302496 [32032/60000]
loss: 2.303416 [35232/60000]
loss: 2.303174 [38432/60000]
loss: 2.302906 [41632/60000]
loss: 2.303993 [44832/60000]
loss: 2.302703 [48032/60000]
loss: 2.301974 [51232/60000]
```

```
loss: 2.302108 [54432/60000]
loss: 2.301692 [57632/60000]
One epoch takes 46.374499797821045
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302650

Epoch 15
-------------------------------
loss: 2.306055 [   32/60000]
loss: 2.303955 [ 3232/60000]
loss: 2.299403 [ 6432/60000]
loss: 2.302818 [ 9632/60000]
loss: 2.300344 [12832/60000]
loss: 2.305896 [16032/60000]
loss: 2.304557 [19232/60000]
loss: 2.302475 [22432/60000]
loss: 2.299032 [25632/60000]
loss: 2.300082 [28832/60000]
loss: 2.303813 [32032/60000]
loss: 2.303083 [35232/60000]
loss: 2.301854 [38432/60000]
loss: 2.304225 [41632/60000]
loss: 2.303084 [44832/60000]
loss: 2.304759 [48032/60000]
loss: 2.301450 [51232/60000]
loss: 2.302706 [54432/60000]
loss: 2.301679 [57632/60000]
One epoch takes 49.86219811439514
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302597

Epoch 16
-------------------------------
loss: 2.303358 [   32/60000]
loss: 2.302842 [ 3232/60000]
loss: 2.300082 [ 6432/60000]
loss: 2.302017 [ 9632/60000]
loss: 2.301737 [12832/60000]
loss: 2.300736 [16032/60000]
loss: 2.301491 [19232/60000]
loss: 2.301299 [22432/60000]
loss: 2.302703 [25632/60000]
loss: 2.302787 [28832/60000]
loss: 2.303451 [32032/60000]
loss: 2.303405 [35232/60000]
loss: 2.301699 [38432/60000]
loss: 2.300470 [41632/60000]
loss: 2.300997 [44832/60000]
loss: 2.304200 [48032/60000]
loss: 2.302576 [51232/60000]
loss: 2.302593 [54432/60000]
loss: 2.299631 [57632/60000]
One epoch takes 45.18250513076782
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302632

Epoch 17
-------------------------------
loss: 2.299719 [   32/60000]
loss: 2.303190 [ 3232/60000]
loss: 2.302655 [ 6432/60000]
loss: 2.300980 [ 9632/60000]
loss: 2.303481 [12832/60000]
loss: 2.303086 [16032/60000]
loss: 2.303561 [19232/60000]
loss: 2.303083 [22432/60000]
loss: 2.301856 [25632/60000]
loss: 2.303347 [28832/60000]
loss: 2.302494 [32032/60000]
```

```
loss: 2.302790 [35232/60000]
loss: 2.302163 [38432/60000]
loss: 2.304260 [41632/60000]
loss: 2.301374 [44832/60000]
loss: 2.302703 [48032/60000]
loss: 2.303322 [51232/60000]
loss: 2.302617 [54432/60000]
loss: 2.303611 [57632/60000]
One epoch takes 42.37205410003662
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302593

Epoch 18
-------------------------------
loss: 2.302881 [   32/60000]
loss: 2.303006 [ 3232/60000]
loss: 2.301985 [ 6432/60000]
loss: 2.300565 [ 9632/60000]
loss: 2.300259 [12832/60000]
loss: 2.304740 [16032/60000]
loss: 2.303868 [19232/60000]
loss: 2.305003 [22432/60000]
loss: 2.303434 [25632/60000]
loss: 2.298970 [28832/60000]
loss: 2.302978 [32032/60000]
loss: 2.303702 [35232/60000]
loss: 2.306760 [38432/60000]
loss: 2.299605 [41632/60000]
loss: 2.302746 [44832/60000]
loss: 2.304110 [48032/60000]
loss: 2.303326 [51232/60000]
loss: 2.302237 [54432/60000]
loss: 2.305492 [57632/60000]
One epoch takes 42.49147081375122
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302628

Epoch 19
-------------------------------
loss: 2.304023 [   32/60000]
loss: 2.305280 [ 3232/60000]
loss: 2.303371 [ 6432/60000]
loss: 2.305037 [ 9632/60000]
loss: 2.302689 [12832/60000]
loss: 2.302989 [16032/60000]
loss: 2.300632 [19232/60000]
loss: 2.303255 [22432/60000]
loss: 2.302095 [25632/60000]
loss: 2.302654 [28832/60000]
loss: 2.302899 [32032/60000]
loss: 2.302691 [35232/60000]
loss: 2.300919 [38432/60000]
loss: 2.303025 [41632/60000]
loss: 2.302919 [44832/60000]
loss: 2.302286 [48032/60000]
loss: 2.302188 [51232/60000]
loss: 2.302535 [54432/60000]
loss: 2.303299 [57632/60000]
One epoch takes 42.707270860672
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302593

Epoch 20
-------------------------------
loss: 2.302267 [   32/60000]
loss: 2.302182 [ 3232/60000]
loss: 2.302589 [ 6432/60000]
loss: 2.302123 [ 9632/60000]
loss: 2.302738 [12832/60000]
```

```
loss: 2.302364 [16032/60000]
loss: 2.302001 [19232/60000]
loss: 2.302267 [22432/60000]
loss: 2.301513 [25632/60000]
loss: 2.302226 [28832/60000]
loss: 2.301826 [32032/60000]
loss: 2.304203 [35232/60000]
loss: 2.305092 [38432/60000]
loss: 2.302969 [41632/60000]
loss: 2.301401 [44832/60000]
loss: 2.303640 [48032/60000]
loss: 2.303542 [51232/60000]
loss: 2.302414 [54432/60000]
loss: 2.303343 [57632/60000]
One epoch takes 42.94412899017334
Test Error:
 Accuracy: 10.0%, Avg loss: 2.302594

Test Error:
 Accuracy: 10.0%, Avg loss: 2.302594

Training done!
```

Problem B.2. ResNet

Now, keep the general structure in B.1 almost unchanged, but add some skip connections (ResNet structure). Try to achieve 89% accuracy again.

Hint: To add a skip connection when calling a module in `forward`, instead of:

```
y = net1(x)
```

use:

```
y = net1(x) + x
```

The input and output of each middle block have the same shape, so each copy of `self.mid_block` in your DeeperCNN can use a skip connection. Try to find where else you can add this skip connection.

Use the single-channel FashionMNIST input (`in_channels=1`) in B.2 and B.3. As a special case, `self.start_conv(x) + x` is also valid: PyTorch broadcasts the single input channel across the 16 output channels.

```python
class ResCNN(nn.Module):
  def __init__(
    self,
    img_size: int=32,
    in_channels: int=1,
    num_classes: int=10,
  ):
    ######################
    # Your code goes here
    ######################
    super().__init__()
    base_channel = 16

    self.start_conv = nn.Sequential(
        *gen_double_conv(in_channels, base_channel),
    )
    self.start_ds = nn.Sequential(nn.AvgPool2d(2))

    # 每个块都重新创建，不共享权重
    self.mid_block = nn.ModuleList([
        nn.Sequential(
```

```
            nn.Conv2d(base_channel, base_channel * 2, 3, padding=1),
            nn.ReLU(),
            nn.AvgPool2d(2),
            nn.Conv2d(base_channel * 2, base_channel * 2, 3, padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(base_channel * 2, base_channel, 2, stride=2),
            nn.ReLU(),
        ) for _ in range(8)
    ])

    self.end_conv1 = nn.Sequential(
        *gen_double_conv(base_channel, base_channel * 2)
    )
    self.end_ds1 = nn.Sequential(nn.AvgPool2d(2))
    self.end_conv2 = nn.Sequential(
        *gen_double_conv(base_channel * 2, base_channel * 4)
    )

    n_pooling_layer = 3
    last_layer_size = img_size // (2 ** n_pooling_layer)
    self.end_ds2_linear = nn.Sequential(
        nn.AvgPool2d(2),
        nn.Flatten(),
        nn.Linear(last_layer_size * last_layer_size *
                  base_channel * 4, base_channel * 8),
        nn.ReLU(),
        nn.Linear(base_channel * 8, num_classes)
    )

  def forward(self, x):
    ######################
    # Your code goes here
    ######################
    # 单通道输入会广播到 16 个通道
    after_start = self.start_ds(self.start_conv(x) + x)
    after_mid = after_start
    for block in self.mid_block:
      after_mid = block(after_mid) + after_mid
    output = self.end_ds2_linear(
        self.end_conv2(self.end_ds1(self.end_conv1(after_mid))))
    return output

  def get_n_conv_linear_layers(self) -> int:
    ######################
    # Your code goes here
    ######################
    return sum(isinstance(mod, (nn.ConvTranspose2d, nn.Conv2d, nn.Linear))
               for mod in self.modules())
```

```
#@title Create a model and its optimizer

model = ResCNN()
model.to(device)
loss_fn = nn.CrossEntropyLoss()

epochs = 20   # Number of training epochs
optimizer = torch.optim.Adam(
  model.parameters(),    # All trainable parameters.
  lr=1e-4                # Learning rate
)

print(f'The model contains {model.get_n_conv_linear_layers()} conv or '
      f'linear layers')
```

```
The model contains 32 conv or linear layers
```

Run the training code below.

Now, with the ResNet design, you should reach 89% accuracy again.

```
#@title Run training

for t in range(epochs):
  print(f"Epoch {t+1}\n-------------------------------")
  start_time =time.time()
  _, _ = train_one_epoch(train_loader, model, loss_fn, optimizer, device=device)
  print(f'One epoch takes {time.time() - start_time}')
  _, _ = test_all_samples(test_loader, model, loss_fn, device=device)

test_all_samples(test_loader, model, loss_fn, device=device)

print("Training done!")
```

```
Epoch 1
-------------------------------
loss: 2.296765 [   32/60000]
loss: 0.931875 [ 3232/60000]
loss: 1.230693 [ 6432/60000]
loss: 0.807296 [ 9632/60000]
loss: 0.752062 [12832/60000]
loss: 0.702370 [16032/60000]
loss: 0.789031 [19232/60000]
loss: 1.001689 [22432/60000]
loss: 0.999164 [25632/60000]
loss: 0.989125 [28832/60000]
loss: 0.579765 [32032/60000]
loss: 0.476446 [35232/60000]
loss: 0.696485 [38432/60000]
loss: 0.865864 [41632/60000]
loss: 0.516393 [44832/60000]
loss: 0.516359 [48032/60000]
loss: 0.614260 [51232/60000]
loss: 0.534325 [54432/60000]
loss: 0.577854 [57632/60000]
One epoch takes 44.062565088272095
Test Error:
 Accuracy: 80.7%, Avg loss: 0.528147

Epoch 2
-------------------------------
loss: 0.696008 [   32/60000]
loss: 0.764654 [ 3232/60000]
loss: 0.645739 [ 6432/60000]
loss: 0.260945 [ 9632/60000]
loss: 0.257099 [12832/60000]
loss: 0.363889 [16032/60000]
loss: 0.298390 [19232/60000]
loss: 0.548771 [22432/60000]
loss: 0.324059 [25632/60000]
loss: 0.757530 [28832/60000]
loss: 0.572545 [32032/60000]
loss: 0.300264 [35232/60000]
loss: 0.370264 [38432/60000]
loss: 0.320224 [41632/60000]
loss: 0.340381 [44832/60000]
loss: 0.336412 [48032/60000]
loss: 0.794969 [51232/60000]
loss: 0.429476 [54432/60000]
loss: 0.136025 [57632/60000]
One epoch takes 46.706392765045166
Test Error:
 Accuracy: 84.5%, Avg loss: 0.419461
```

```
Epoch 3
-------------------------------
loss: 0.272282 [   32/60000]
loss: 0.338440 [ 3232/60000]
loss: 0.381464 [ 6432/60000]
loss: 0.371686 [ 9632/60000]
loss: 0.385492 [12832/60000]
loss: 0.247950 [16032/60000]
loss: 0.275981 [19232/60000]
loss: 0.214435 [22432/60000]
loss: 0.471829 [25632/60000]
loss: 0.394606 [28832/60000]
loss: 0.204245 [32032/60000]
loss: 0.234013 [35232/60000]
loss: 0.262460 [38432/60000]
loss: 0.512999 [41632/60000]
loss: 0.322553 [44832/60000]
loss: 0.378132 [48032/60000]
loss: 0.518999 [51232/60000]
loss: 0.226357 [54432/60000]
loss: 0.354065 [57632/60000]
One epoch takes 45.46146297454834
Test Error:
 Accuracy: 85.2%, Avg loss: 0.400578

Epoch 4
-------------------------------
loss: 0.142700 [   32/60000]
loss: 0.216143 [ 3232/60000]
loss: 0.342964 [ 6432/60000]
loss: 0.427505 [ 9632/60000]
loss: 0.249961 [12832/60000]
loss: 0.387092 [16032/60000]
loss: 0.152451 [19232/60000]
loss: 0.586878 [22432/60000]
loss: 0.349755 [25632/60000]
loss: 0.319264 [28832/60000]
loss: 0.530101 [32032/60000]
loss: 0.184168 [35232/60000]
loss: 0.366512 [38432/60000]
loss: 0.299740 [41632/60000]
loss: 0.181686 [44832/60000]
loss: 0.391055 [48032/60000]
loss: 0.504934 [51232/60000]
loss: 0.176958 [54432/60000]
loss: 0.339527 [57632/60000]
One epoch takes 45.06798195838928
Test Error:
 Accuracy: 87.3%, Avg loss: 0.341308

Epoch 5
-------------------------------
loss: 0.244502 [   32/60000]
loss: 0.307164 [ 3232/60000]
loss: 0.432493 [ 6432/60000]
loss: 0.316708 [ 9632/60000]
loss: 0.298963 [12832/60000]
loss: 0.266564 [16032/60000]
loss: 0.555882 [19232/60000]
loss: 0.277777 [22432/60000]
loss: 0.233298 [25632/60000]
loss: 0.601316 [28832/60000]
loss: 0.232755 [32032/60000]
loss: 0.300566 [35232/60000]
loss: 0.424236 [38432/60000]
loss: 0.396017 [41632/60000]
loss: 0.329392 [44832/60000]
loss: 0.488297 [48032/60000]
```

```
loss: 0.160250 [51232/60000]
loss: 0.242727 [54432/60000]
loss: 0.489475 [57632/60000]
One epoch takes 45.06489014625549
Test Error:
 Accuracy: 88.1%, Avg loss: 0.316492

Epoch 6
-------------------------------
loss: 0.154209 [   32/60000]
loss: 0.104283 [ 3232/60000]
loss: 0.131269 [ 6432/60000]
loss: 0.267890 [ 9632/60000]
loss: 0.279260 [12832/60000]
loss: 0.206207 [16032/60000]
loss: 0.251942 [19232/60000]
loss: 0.337986 [22432/60000]
loss: 0.132978 [25632/60000]
loss: 0.404689 [28832/60000]
loss: 0.400272 [32032/60000]
loss: 0.155539 [35232/60000]
loss: 0.199356 [38432/60000]
loss: 0.504648 [41632/60000]
loss: 0.328786 [44832/60000]
loss: 0.263118 [48032/60000]
loss: 0.098553 [51232/60000]
loss: 0.161020 [54432/60000]
loss: 0.198627 [57632/60000]
One epoch takes 44.86375689506531
Test Error:
 Accuracy: 88.1%, Avg loss: 0.315761

Epoch 7
-------------------------------
loss: 0.141064 [   32/60000]
loss: 0.166132 [ 3232/60000]
loss: 0.209470 [ 6432/60000]
loss: 0.185830 [ 9632/60000]
loss: 0.348788 [12832/60000]
loss: 0.489718 [16032/60000]
loss: 0.325888 [19232/60000]
loss: 0.078997 [22432/60000]
loss: 0.167262 [25632/60000]
loss: 0.317347 [28832/60000]
loss: 0.338206 [32032/60000]
loss: 0.374370 [35232/60000]
loss: 0.148562 [38432/60000]
loss: 0.127825 [41632/60000]
loss: 0.432215 [44832/60000]
loss: 0.218476 [48032/60000]
loss: 0.112370 [51232/60000]
loss: 0.219556 [54432/60000]
loss: 0.323639 [57632/60000]
One epoch takes 45.23512387275696
Test Error:
 Accuracy: 88.8%, Avg loss: 0.300738

Epoch 8
-------------------------------
loss: 0.339533 [   32/60000]
loss: 0.160139 [ 3232/60000]
loss: 0.268168 [ 6432/60000]
loss: 0.441581 [ 9632/60000]
loss: 0.150758 [12832/60000]
loss: 0.255300 [16032/60000]
loss: 0.281862 [19232/60000]
loss: 0.167186 [22432/60000]
loss: 0.185175 [25632/60000]
loss: 0.050261 [28832/60000]
```

```
loss: 0.176701 [32032/60000]
loss: 0.152780 [35232/60000]
loss: 0.277368 [38432/60000]
loss: 0.190888 [41632/60000]
loss: 0.367649 [44832/60000]
loss: 0.135009 [48032/60000]
loss: 0.372243 [51232/60000]
loss: 0.328943 [54432/60000]
loss: 0.186465 [57632/60000]
One epoch takes 45.96007227897644
Test Error:
 Accuracy: 89.3%, Avg loss: 0.288559

Epoch 9
-------------------------------
loss: 0.428769 [   32/60000]
loss: 0.408301 [ 3232/60000]
loss: 0.230959 [ 6432/60000]
loss: 0.247208 [ 9632/60000]
loss: 0.244794 [12832/60000]
loss: 0.079295 [16032/60000]
loss: 0.075277 [19232/60000]
loss: 0.141753 [22432/60000]
loss: 0.318312 [25632/60000]
loss: 0.349921 [28832/60000]
loss: 0.179269 [32032/60000]
loss: 0.433992 [35232/60000]
loss: 0.325125 [38432/60000]
loss: 0.285050 [41632/60000]
loss: 0.295319 [44832/60000]
loss: 0.156831 [48032/60000]
loss: 0.275494 [51232/60000]
loss: 0.366989 [54432/60000]
loss: 0.229339 [57632/60000]
One epoch takes 46.22822022438049
Test Error:
 Accuracy: 89.4%, Avg loss: 0.285155

Epoch 10
-------------------------------
loss: 0.261954 [   32/60000]
loss: 0.231852 [ 3232/60000]
loss: 0.208066 [ 6432/60000]
loss: 0.279988 [ 9632/60000]
loss: 0.094314 [12832/60000]
loss: 0.147388 [16032/60000]
loss: 0.469426 [19232/60000]
loss: 0.217471 [22432/60000]
loss: 0.230060 [25632/60000]
loss: 0.249180 [28832/60000]
loss: 0.183003 [32032/60000]
loss: 0.162665 [35232/60000]
loss: 0.182257 [38432/60000]
loss: 0.092227 [41632/60000]
loss: 0.352095 [44832/60000]
loss: 0.258881 [48032/60000]
loss: 0.317060 [51232/60000]
loss: 0.170157 [54432/60000]
loss: 0.438320 [57632/60000]
One epoch takes 45.28334283828735
Test Error:
 Accuracy: 89.0%, Avg loss: 0.291667

Epoch 11
-------------------------------
loss: 0.133226 [   32/60000]
loss: 0.064739 [ 3232/60000]
loss: 0.079467 [ 6432/60000]
loss: 0.256973 [ 9632/60000]
```

```
loss: 0.308964 [12832/60000]
loss: 0.237806 [16032/60000]
loss: 0.440869 [19232/60000]
loss: 0.320036 [22432/60000]
loss: 0.161926 [25632/60000]
loss: 0.180958 [28832/60000]
loss: 0.151746 [32032/60000]
loss: 0.226683 [35232/60000]
loss: 0.049907 [38432/60000]
loss: 0.275972 [41632/60000]
loss: 0.179213 [44832/60000]
loss: 0.600434 [48032/60000]
loss: 0.213796 [51232/60000]
loss: 0.142334 [54432/60000]
loss: 0.246341 [57632/60000]
One epoch takes 45.156299114227295
Test Error:
 Accuracy: 90.3%, Avg loss: 0.262259

Epoch 12
-------------------------------
loss: 0.090441 [   32/60000]
loss: 0.091782 [ 3232/60000]
loss: 0.165950 [ 6432/60000]
loss: 0.118703 [ 9632/60000]
loss: 0.293812 [12832/60000]
loss: 0.271486 [16032/60000]
loss: 0.154705 [19232/60000]
loss: 0.029969 [22432/60000]
loss: 0.059907 [25632/60000]
loss: 0.371760 [28832/60000]
loss: 0.290581 [32032/60000]
loss: 0.167450 [35232/60000]
loss: 0.116317 [38432/60000]
loss: 0.389734 [41632/60000]
loss: 0.252929 [44832/60000]
loss: 0.245249 [48032/60000]
loss: 0.290276 [51232/60000]
loss: 0.189832 [54432/60000]
loss: 0.195092 [57632/60000]
One epoch takes 51.04285526275635
Test Error:
 Accuracy: 90.6%, Avg loss: 0.264393

Epoch 13
-------------------------------
loss: 0.052447 [   32/60000]
loss: 0.163651 [ 3232/60000]
loss: 0.221778 [ 6432/60000]
loss: 0.195596 [ 9632/60000]
loss: 0.289616 [12832/60000]
loss: 0.182390 [16032/60000]
loss: 0.146166 [19232/60000]
loss: 0.354144 [22432/60000]
loss: 0.126242 [25632/60000]
loss: 0.124837 [28832/60000]
loss: 0.166043 [32032/60000]
loss: 0.269024 [35232/60000]
loss: 0.358709 [38432/60000]
loss: 0.245002 [41632/60000]
loss: 0.302949 [44832/60000]
loss: 0.085982 [48032/60000]
loss: 0.205156 [51232/60000]
loss: 0.158213 [54432/60000]
loss: 0.112079 [57632/60000]
One epoch takes 46.323559045791626
Test Error:
 Accuracy: 90.0%, Avg loss: 0.272053
```

```
Epoch 14
-------------------------------
loss: 0.218936 [   32/60000]
loss: 0.110264 [ 3232/60000]
loss: 0.485474 [ 6432/60000]
loss: 0.308974 [ 9632/60000]
loss: 0.316069 [12832/60000]
loss: 0.128661 [16032/60000]
loss: 0.117912 [19232/60000]
loss: 0.182652 [22432/60000]
loss: 0.250446 [25632/60000]
loss: 0.326422 [28832/60000]
loss: 0.131717 [32032/60000]
loss: 0.221338 [35232/60000]
loss: 0.136851 [38432/60000]
loss: 0.442626 [41632/60000]
loss: 0.403236 [44832/60000]
loss: 0.153157 [48032/60000]
loss: 0.254668 [51232/60000]
loss: 0.162799 [54432/60000]
loss: 0.131152 [57632/60000]
One epoch takes 46.03508400917053
Test Error:
 Accuracy: 90.4%, Avg loss: 0.268407

Epoch 15
-------------------------------
loss: 0.377930 [   32/60000]
loss: 0.083041 [ 3232/60000]
loss: 0.674705 [ 6432/60000]
loss: 0.079224 [ 9632/60000]
loss: 0.404944 [12832/60000]
loss: 0.219691 [16032/60000]
loss: 0.389828 [19232/60000]
loss: 0.194853 [22432/60000]
loss: 0.284619 [25632/60000]
loss: 0.340026 [28832/60000]
loss: 0.110104 [32032/60000]
loss: 0.253594 [35232/60000]
loss: 0.236945 [38432/60000]
loss: 0.210052 [41632/60000]
loss: 0.210307 [44832/60000]
loss: 0.280338 [48032/60000]
loss: 0.314896 [51232/60000]
loss: 0.130136 [54432/60000]
loss: 0.116435 [57632/60000]
One epoch takes 44.99865698814392
Test Error:
 Accuracy: 90.5%, Avg loss: 0.269538

Epoch 16
-------------------------------
loss: 0.524144 [   32/60000]
loss: 0.161898 [ 3232/60000]
loss: 0.109778 [ 6432/60000]
loss: 0.162823 [ 9632/60000]
loss: 0.155535 [12832/60000]
loss: 0.461489 [16032/60000]
loss: 0.311370 [19232/60000]
loss: 0.140223 [22432/60000]
loss: 0.157639 [25632/60000]
loss: 0.256785 [28832/60000]
loss: 0.170624 [32032/60000]
loss: 0.175795 [35232/60000]
loss: 0.182434 [38432/60000]
loss: 0.218225 [41632/60000]
loss: 0.091107 [44832/60000]
loss: 0.166624 [48032/60000]
loss: 0.061516 [51232/60000]
```

```
loss: 0.102895 [54432/60000]
loss: 0.075463 [57632/60000]
One epoch takes 46.154634952545166
Test Error:
 Accuracy: 90.7%, Avg loss: 0.259455

Epoch 17
-------------------------------
loss: 0.123360 [   32/60000]
loss: 0.160280 [ 3232/60000]
loss: 0.200574 [ 6432/60000]
loss: 0.139520 [ 9632/60000]
loss: 0.129981 [12832/60000]
loss: 0.280635 [16032/60000]
loss: 0.173219 [19232/60000]
loss: 0.139039 [22432/60000]
loss: 0.096252 [25632/60000]
loss: 0.077099 [28832/60000]
loss: 0.255637 [32032/60000]
loss: 0.150250 [35232/60000]
loss: 0.018193 [38432/60000]
loss: 0.131893 [41632/60000]
loss: 0.182789 [44832/60000]
loss: 0.097147 [48032/60000]
loss: 0.308785 [51232/60000]
loss: 0.167623 [54432/60000]
loss: 0.249038 [57632/60000]
One epoch takes 52.79384994506836
Test Error:
 Accuracy: 90.5%, Avg loss: 0.272396

Epoch 18
-------------------------------
loss: 0.090636 [   32/60000]
loss: 0.114931 [ 3232/60000]
loss: 0.206981 [ 6432/60000]
loss: 0.238898 [ 9632/60000]
loss: 0.066333 [12832/60000]
loss: 0.254089 [16032/60000]
loss: 0.136143 [19232/60000]
loss: 0.217274 [22432/60000]
loss: 0.078761 [25632/60000]
loss: 0.033485 [28832/60000]
loss: 0.070367 [32032/60000]
loss: 0.134035 [35232/60000]
loss: 0.212298 [38432/60000]
loss: 0.139267 [41632/60000]
loss: 0.059856 [44832/60000]
loss: 0.169848 [48032/60000]
loss: 0.343871 [51232/60000]
loss: 0.107680 [54432/60000]
loss: 0.213380 [57632/60000]
One epoch takes 54.89737510681152
Test Error:
 Accuracy: 90.8%, Avg loss: 0.260631

Epoch 19
-------------------------------
loss: 0.131369 [   32/60000]
loss: 0.170125 [ 3232/60000]
loss: 0.115856 [ 6432/60000]
loss: 0.446654 [ 9632/60000]
loss: 0.023760 [12832/60000]
loss: 0.252235 [16032/60000]
loss: 0.201755 [19232/60000]
loss: 0.133647 [22432/60000]
loss: 0.085387 [25632/60000]
loss: 0.278810 [28832/60000]
loss: 0.070450 [32032/60000]
```

```
loss: 0.117469 [35232/60000]
loss: 0.277610 [38432/60000]
loss: 0.050209 [41632/60000]
loss: 0.093624 [44832/60000]
loss: 0.437816 [48032/60000]
loss: 0.170329 [51232/60000]
loss: 0.039348 [54432/60000]
loss: 0.173185 [57632/60000]
One epoch takes 58.97937822341919
Test Error:
 Accuracy: 90.4%, Avg loss: 0.280152

Epoch 20
-------------------------------
loss: 0.133862 [   32/60000]
loss: 0.179525 [ 3232/60000]
loss: 0.086397 [ 6432/60000]
loss: 0.154242 [ 9632/60000]
loss: 0.244362 [12832/60000]
loss: 0.135970 [16032/60000]
loss: 0.240852 [19232/60000]
loss: 0.126760 [22432/60000]
loss: 0.157898 [25632/60000]
loss: 0.100272 [28832/60000]
loss: 0.139257 [32032/60000]
loss: 0.083724 [35232/60000]
loss: 0.066751 [38432/60000]
loss: 0.223013 [41632/60000]
loss: 0.188167 [44832/60000]
loss: 0.054693 [48032/60000]
loss: 0.530676 [51232/60000]
loss: 0.067058 [54432/60000]
loss: 0.154373 [57632/60000]
One epoch takes 58.308436155319214
Test Error:
 Accuracy: 90.7%, Avg loss: 0.269940

Test Error:
 Accuracy: 90.7%, Avg loss: 0.269940

Training done!
```

Problem B.3. Residual connections with different dimensions

Now, keep the general structure in B.2 almost unchanged, but add a skip connection to `self.end_conv1`.

The class below inherits from your `ResCNN` in B.2.

Hint: `super().__init__(...)` already creates the layers from B.2, so you do not need to define them again. Run the B.2 class definition first. You do not need to run its training code.

In B.2, the middle blocks have matching input and output shapes. However, `self.end_conv1` changes the number of channels from 16 to 32. Please transform the shortcut input so that it can be added to the output of this block.

Move the final ReLU of this block so that it is applied after the addition. Then run the training code below.

Hint: Use a `1x1` convolution with `stride=1` to match the number of channels while keeping the spatial dimensions unchanged.

```
class ProjectionResCNN(ResCNN):
  def __init__(
    self,
    img_size: int=32,
```

```
    in_channels: int=1,
    num_classes: int=10,
  ):
    super().__init__(img_size, in_channels, num_classes)
    ######################
    # Your code goes here
    ######################
    self.shortcut = nn.Conv2d(16, 32, 1, stride=1)
    # 把最后一个 ReLU 移到残差相加之后
    self.end_relu = self.end_conv1[-1]
    self.end_conv1 = self.end_conv1[:-1]

  def forward(self, x):
    ######################
    # Your code goes here
    ######################
    after_start = self.start_ds(self.start_conv(x) + x)
    after_mid = after_start
    for block in self.mid_block:
      after_mid = block(after_mid) + after_mid
    after_end1 = self.end_relu(
        self.end_conv1(after_mid) + self.shortcut(after_mid))
    output = self.end_ds2_linear(
        self.end_conv2(self.end_ds1(after_end1)))
    return output

  def get_n_conv_linear_layers(self) -> int:
    ######################
    # Your code goes here
    ######################
    return sum(isinstance(mod, (nn.ConvTranspose2d, nn.Conv2d, nn.Linear))
               for mod in self.modules())
```

```
#@title Create a model and its optimizer

model = ProjectionResCNN()
model.to(device)
loss_fn = nn.CrossEntropyLoss()

epochs = 20   # Number of training epochs
optimizer = torch.optim.Adam(
  model.parameters(),    # All trainable parameters.
  lr=1e-4                # Learning rate
)

print(f'The model contains {model.get_n_conv_linear_layers()} conv or '
      f'linear layers')
```

```
The model contains 33 conv or linear layers
```

Run the training code below.

```
#@title Run training

for t in range(epochs):
  print(f"Epoch {t+1}\n-------------------------------")
  start_time =time.time()
  _, _ = train_one_epoch(train_loader, model, loss_fn, optimizer, device=device)
  print(f'One epoch takes {time.time() - start_time}')
  _, _ = test_all_samples(test_loader, model, loss_fn, device=device)

test_all_samples(test_loader, model, loss_fn, device=device)
```

```
print("Training done!")
```

```
Epoch 1
-------------------------------
loss: 2.309994 [   32/60000]
loss: 0.811989 [ 3232/60000]
loss: 0.631085 [ 6432/60000]
loss: 0.935692 [ 9632/60000]
loss: 0.683242 [12832/60000]
loss: 0.656229 [16032/60000]
loss: 0.576167 [19232/60000]
loss: 0.982072 [22432/60000]
loss: 0.632845 [25632/60000]
loss: 0.812827 [28832/60000]
loss: 0.495586 [32032/60000]
loss: 0.362786 [35232/60000]
loss: 0.338734 [38432/60000]
loss: 0.467352 [41632/60000]
loss: 0.743112 [44832/60000]
loss: 0.409393 [48032/60000]
loss: 0.939494 [51232/60000]
loss: 0.311873 [54432/60000]
loss: 0.604337 [57632/60000]
One epoch takes 56.79745101928711
Test Error:
 Accuracy: 79.3%, Avg loss: 0.554658

Epoch 2
-------------------------------
loss: 0.565975 [   32/60000]
loss: 0.391321 [ 3232/60000]
loss: 0.409992 [ 6432/60000]
loss: 0.265380 [ 9632/60000]
loss: 0.445770 [12832/60000]
loss: 0.491380 [16032/60000]
loss: 0.461512 [19232/60000]
loss: 0.453100 [22432/60000]
loss: 0.329951 [25632/60000]
loss: 0.334306 [28832/60000]
loss: 0.518733 [32032/60000]
loss: 0.386177 [35232/60000]
loss: 0.387634 [38432/60000]
loss: 0.570499 [41632/60000]
loss: 0.362016 [44832/60000]
loss: 0.667638 [48032/60000]
loss: 0.636357 [51232/60000]
loss: 0.506559 [54432/60000]
loss: 0.365138 [57632/60000]
One epoch takes 58.614436864852905
Test Error:
 Accuracy: 84.7%, Avg loss: 0.408161

Epoch 3
-------------------------------
loss: 0.488421 [   32/60000]
loss: 0.360738 [ 3232/60000]
loss: 0.571352 [ 6432/60000]
loss: 0.440373 [ 9632/60000]
loss: 0.319944 [12832/60000]
loss: 0.311694 [16032/60000]
loss: 0.265959 [19232/60000]
loss: 0.730665 [22432/60000]
loss: 0.172393 [25632/60000]
loss: 0.148629 [28832/60000]
loss: 0.240713 [32032/60000]
loss: 0.209620 [35232/60000]
loss: 0.244154 [38432/60000]
```

```
loss: 0.480941 [41632/60000]
loss: 0.251231 [44832/60000]
loss: 0.415703 [48032/60000]
loss: 0.243008 [51232/60000]
loss: 0.361373 [54432/60000]
loss: 0.516021 [57632/60000]
One epoch takes 55.66812992095947
Test Error:
 Accuracy: 86.6%, Avg loss: 0.366078

Epoch 4
-------------------------------
loss: 0.333075 [   32/60000]
loss: 0.232853 [ 3232/60000]
loss: 0.226412 [ 6432/60000]
loss: 0.478667 [ 9632/60000]
loss: 0.436788 [12832/60000]
loss: 0.101568 [16032/60000]
loss: 0.392272 [19232/60000]
loss: 0.276673 [22432/60000]
loss: 0.367036 [25632/60000]
loss: 0.254217 [28832/60000]
loss: 0.212789 [32032/60000]
loss: 0.399645 [35232/60000]
loss: 0.296997 [38432/60000]
loss: 0.206915 [41632/60000]
loss: 0.384696 [44832/60000]
loss: 0.250941 [48032/60000]
loss: 0.424148 [51232/60000]
loss: 0.316554 [54432/60000]
loss: 0.267164 [57632/60000]
One epoch takes 56.5135338306427
Test Error:
 Accuracy: 87.7%, Avg loss: 0.334359

Epoch 5
-------------------------------
loss: 0.273662 [   32/60000]
loss: 0.233429 [ 3232/60000]
loss: 0.285546 [ 6432/60000]
loss: 0.062606 [ 9632/60000]
loss: 0.206521 [12832/60000]
loss: 0.357036 [16032/60000]
loss: 0.284155 [19232/60000]
loss: 0.195111 [22432/60000]
loss: 0.224976 [25632/60000]
loss: 0.250130 [28832/60000]
loss: 0.313611 [32032/60000]
loss: 0.348820 [35232/60000]
loss: 0.419255 [38432/60000]
loss: 0.323933 [41632/60000]
loss: 0.162421 [44832/60000]
loss: 0.190382 [48032/60000]
loss: 0.308139 [51232/60000]
loss: 0.588823 [54432/60000]
loss: 0.157999 [57632/60000]
One epoch takes 52.47193098068237
Test Error:
 Accuracy: 87.3%, Avg loss: 0.337050

Epoch 6
-------------------------------
loss: 0.422595 [   32/60000]
loss: 0.319065 [ 3232/60000]
loss: 0.366792 [ 6432/60000]
loss: 0.514252 [ 9632/60000]
loss: 0.205565 [12832/60000]
loss: 0.208414 [16032/60000]
loss: 0.263209 [19232/60000]
```

```
loss: 0.212227 [22432/60000]
loss: 0.240014 [25632/60000]
loss: 0.357652 [28832/60000]
loss: 0.158137 [32032/60000]
loss: 0.472190 [35232/60000]
loss: 0.318014 [38432/60000]
loss: 0.230962 [41632/60000]
loss: 0.065858 [44832/60000]
loss: 0.307216 [48032/60000]
loss: 0.072840 [51232/60000]
loss: 0.097950 [54432/60000]
loss: 0.220045 [57632/60000]
One epoch takes 52.9784300327301
Test Error:
 Accuracy: 89.0%, Avg loss: 0.304076

Epoch 7
-------------------------------
loss: 0.235771 [   32/60000]
loss: 0.284677 [ 3232/60000]
loss: 0.215676 [ 6432/60000]
loss: 0.103710 [ 9632/60000]
loss: 0.274029 [12832/60000]
loss: 0.400053 [16032/60000]
loss: 0.258760 [19232/60000]
loss: 0.336938 [22432/60000]
loss: 0.527682 [25632/60000]
loss: 0.141918 [28832/60000]
loss: 0.136095 [32032/60000]
loss: 0.279020 [35232/60000]
loss: 0.252559 [38432/60000]
loss: 0.260319 [41632/60000]
loss: 0.179681 [44832/60000]
loss: 0.383046 [48032/60000]
loss: 0.231115 [51232/60000]
loss: 0.353809 [54432/60000]
loss: 0.260729 [57632/60000]
One epoch takes 54.5621120929718
Test Error:
 Accuracy: 88.9%, Avg loss: 0.303021

Epoch 8
-------------------------------
loss: 0.162977 [   32/60000]
loss: 0.323877 [ 3232/60000]
loss: 0.264558 [ 6432/60000]
loss: 0.418124 [ 9632/60000]
loss: 0.304015 [12832/60000]
loss: 0.577612 [16032/60000]
loss: 0.269203 [19232/60000]
loss: 0.538251 [22432/60000]
loss: 0.404225 [25632/60000]
loss: 0.420490 [28832/60000]
loss: 0.336673 [32032/60000]
loss: 0.323736 [35232/60000]
loss: 0.257406 [38432/60000]
loss: 0.202674 [41632/60000]
loss: 0.232641 [44832/60000]
loss: 0.127089 [48032/60000]
loss: 0.190281 [51232/60000]
loss: 0.276316 [54432/60000]
loss: 0.431594 [57632/60000]
One epoch takes 56.07254910469055
Test Error:
 Accuracy: 89.6%, Avg loss: 0.284802

Epoch 9
-------------------------------
loss: 0.178321 [   32/60000]
```

```
loss: 0.071383 [ 3232/60000]
loss: 0.241964 [ 6432/60000]
loss: 0.257896 [ 9632/60000]
loss: 0.138135 [12832/60000]
loss: 0.249025 [16032/60000]
loss: 0.193026 [19232/60000]
loss: 0.171321 [22432/60000]
loss: 0.122280 [25632/60000]
loss: 0.175471 [28832/60000]
loss: 0.137871 [32032/60000]
loss: 0.525904 [35232/60000]
loss: 0.068674 [38432/60000]
loss: 0.115639 [41632/60000]
loss: 0.315527 [44832/60000]
loss: 0.161607 [48032/60000]
loss: 0.232292 [51232/60000]
loss: 0.181367 [54432/60000]
loss: 0.318339 [57632/60000]
One epoch takes 55.2584490776062
Test Error:
 Accuracy: 89.9%, Avg loss: 0.266660

Epoch 10
-------------------------------
loss: 0.149305 [   32/60000]
loss: 0.142549 [ 3232/60000]
loss: 0.126036 [ 6432/60000]
loss: 0.291022 [ 9632/60000]
loss: 0.129190 [12832/60000]
loss: 0.212399 [16032/60000]
loss: 0.161750 [19232/60000]
loss: 0.163639 [22432/60000]
loss: 0.139729 [25632/60000]
loss: 0.171596 [28832/60000]
loss: 0.304050 [32032/60000]
loss: 0.283493 [35232/60000]
loss: 0.306554 [38432/60000]
loss: 0.130911 [41632/60000]
loss: 0.383692 [44832/60000]
loss: 0.208210 [48032/60000]
loss: 0.095454 [51232/60000]
loss: 0.317411 [54432/60000]
loss: 0.103686 [57632/60000]
One epoch takes 57.19799613952637
Test Error:
 Accuracy: 89.4%, Avg loss: 0.286013

Epoch 11
-------------------------------
loss: 0.222337 [   32/60000]
loss: 0.082771 [ 3232/60000]
loss: 0.159494 [ 6432/60000]
loss: 0.192152 [ 9632/60000]
loss: 0.103606 [12832/60000]
loss: 0.129289 [16032/60000]
loss: 0.290327 [19232/60000]
loss: 0.407884 [22432/60000]
loss: 0.396839 [25632/60000]
loss: 0.347762 [28832/60000]
loss: 0.091473 [32032/60000]
loss: 0.300680 [35232/60000]
loss: 0.182123 [38432/60000]
loss: 0.515340 [41632/60000]
loss: 0.263562 [44832/60000]
loss: 0.135927 [48032/60000]
loss: 0.204063 [51232/60000]
loss: 0.014414 [54432/60000]
loss: 0.176434 [57632/60000]
One epoch takes 61.24990892410278
```

```
Test Error:
 Accuracy: 89.6%, Avg loss: 0.284196

Epoch 12
-------------------------------
loss: 0.315712 [   32/60000]
loss: 0.149114 [ 3232/60000]
loss: 0.219298 [ 6432/60000]
loss: 0.215509 [ 9632/60000]
loss: 0.148139 [12832/60000]
loss: 0.156877 [16032/60000]
loss: 0.165050 [19232/60000]
loss: 0.122015 [22432/60000]
loss: 0.310256 [25632/60000]
loss: 0.388597 [28832/60000]
loss: 0.155954 [32032/60000]
loss: 0.042849 [35232/60000]
loss: 0.247436 [38432/60000]
loss: 0.142726 [41632/60000]
loss: 0.113295 [44832/60000]
loss: 0.211682 [48032/60000]
loss: 0.281865 [51232/60000]
loss: 0.057876 [54432/60000]
loss: 0.252331 [57632/60000]
One epoch takes 57.78478002548218
Test Error:
 Accuracy: 90.5%, Avg loss: 0.266531

Epoch 13
-------------------------------
loss: 0.082226 [   32/60000]
loss: 0.055084 [ 3232/60000]
loss: 0.164085 [ 6432/60000]
loss: 0.045183 [ 9632/60000]
loss: 0.242249 [12832/60000]
loss: 0.344846 [16032/60000]
loss: 0.180701 [19232/60000]
loss: 0.466344 [22432/60000]
loss: 0.275018 [25632/60000]
loss: 0.316045 [28832/60000]
loss: 0.073461 [32032/60000]
loss: 0.179186 [35232/60000]
loss: 0.240240 [38432/60000]
loss: 0.151807 [41632/60000]
loss: 0.157302 [44832/60000]
loss: 0.106849 [48032/60000]
loss: 0.278988 [51232/60000]
loss: 0.112645 [54432/60000]
loss: 0.228400 [57632/60000]
One epoch takes 53.90233111381531
Test Error:
 Accuracy: 90.3%, Avg loss: 0.262421

Epoch 14
-------------------------------
loss: 0.219921 [   32/60000]
loss: 0.114324 [ 3232/60000]
loss: 0.372680 [ 6432/60000]
loss: 0.233103 [ 9632/60000]
loss: 0.051912 [12832/60000]
loss: 0.240388 [16032/60000]
loss: 0.142062 [19232/60000]
loss: 0.067705 [22432/60000]
loss: 0.356240 [25632/60000]
loss: 0.249869 [28832/60000]
loss: 0.154880 [32032/60000]
loss: 0.687881 [35232/60000]
loss: 0.255493 [38432/60000]
loss: 0.155435 [41632/60000]
```

```
loss: 0.254335 [44832/60000]
loss: 0.230347 [48032/60000]
loss: 0.117719 [51232/60000]
loss: 0.114719 [54432/60000]
loss: 0.482050 [57632/60000]
One epoch takes 58.672280073165894
Test Error:
 Accuracy: 91.0%, Avg loss: 0.256873

Epoch 15
-------------------------------
loss: 0.153520 [   32/60000]
loss: 0.204174 [ 3232/60000]
loss: 0.219458 [ 6432/60000]
loss: 0.512587 [ 9632/60000]
loss: 0.374323 [12832/60000]
loss: 0.107865 [16032/60000]
loss: 0.132901 [19232/60000]
loss: 0.137340 [22432/60000]
loss: 0.065094 [25632/60000]
loss: 0.165143 [28832/60000]
loss: 0.327182 [32032/60000]
loss: 0.062570 [35232/60000]
loss: 0.118342 [38432/60000]
loss: 0.031106 [41632/60000]
loss: 0.028220 [44832/60000]
loss: 0.270631 [48032/60000]
loss: 0.143998 [51232/60000]
loss: 0.249989 [54432/60000]
loss: 0.150961 [57632/60000]
One epoch takes 52.82853984832764
Test Error:
 Accuracy: 91.0%, Avg loss: 0.264383

Epoch 16
-------------------------------
loss: 0.263284 [   32/60000]
loss: 0.179282 [ 3232/60000]
loss: 0.068844 [ 6432/60000]
loss: 0.284317 [ 9632/60000]
loss: 0.165893 [12832/60000]
loss: 0.154650 [16032/60000]
loss: 0.132675 [19232/60000]
loss: 0.032463 [22432/60000]
loss: 0.126821 [25632/60000]
loss: 0.185607 [28832/60000]
loss: 0.083106 [32032/60000]
loss: 0.104176 [35232/60000]
loss: 0.161108 [38432/60000]
loss: 0.463333 [41632/60000]
loss: 0.103338 [44832/60000]
loss: 0.281562 [48032/60000]
loss: 0.035311 [51232/60000]
loss: 0.188336 [54432/60000]
loss: 0.192393 [57632/60000]
One epoch takes 61.725714921951294
Test Error:
 Accuracy: 90.8%, Avg loss: 0.271247

Epoch 17
-------------------------------
loss: 0.192126 [   32/60000]
loss: 0.170940 [ 3232/60000]
loss: 0.078465 [ 6432/60000]
loss: 0.180570 [ 9632/60000]
loss: 0.070645 [12832/60000]
loss: 0.211434 [16032/60000]
loss: 0.159175 [19232/60000]
loss: 0.226704 [22432/60000]
```

```
loss: 0.163408 [25632/60000]
loss: 0.095623 [28832/60000]
loss: 0.100622 [32032/60000]
loss: 0.140068 [35232/60000]
loss: 0.128227 [38432/60000]
loss: 0.107907 [41632/60000]
loss: 0.133469 [44832/60000]
loss: 0.242748 [48032/60000]
loss: 0.211666 [51232/60000]
loss: 0.100884 [54432/60000]
loss: 0.078495 [57632/60000]
One epoch takes 58.839457988739014
Test Error:
 Accuracy: 90.9%, Avg loss: 0.254291

Epoch 18
-------------------------------
loss: 0.135371 [   32/60000]
loss: 0.118566 [ 3232/60000]
loss: 0.207798 [ 6432/60000]
loss: 0.048615 [ 9632/60000]
loss: 0.383475 [12832/60000]
loss: 0.274702 [16032/60000]
loss: 0.040655 [19232/60000]
loss: 0.126737 [22432/60000]
loss: 0.166667 [25632/60000]
loss: 0.196641 [28832/60000]
loss: 0.081100 [32032/60000]
loss: 0.155816 [35232/60000]
loss: 0.159234 [38432/60000]
loss: 0.162594 [41632/60000]
loss: 0.081473 [44832/60000]
loss: 0.178959 [48032/60000]
loss: 0.145155 [51232/60000]
loss: 0.279789 [54432/60000]
loss: 0.199754 [57632/60000]
One epoch takes 55.08141112327576
Test Error:
 Accuracy: 91.3%, Avg loss: 0.263652

Epoch 19
-------------------------------
loss: 0.076258 [   32/60000]
loss: 0.272331 [ 3232/60000]
loss: 0.100276 [ 6432/60000]
loss: 0.215808 [ 9632/60000]
loss: 0.102065 [12832/60000]
loss: 0.117844 [16032/60000]
loss: 0.262797 [19232/60000]
loss: 0.109190 [22432/60000]
loss: 0.067725 [25632/60000]
loss: 0.432595 [28832/60000]
loss: 0.094487 [32032/60000]
loss: 0.054102 [35232/60000]
loss: 0.084719 [38432/60000]
loss: 0.161739 [41632/60000]
loss: 0.094708 [44832/60000]
loss: 0.311446 [48032/60000]
loss: 0.095230 [51232/60000]
loss: 0.104354 [54432/60000]
loss: 0.166473 [57632/60000]
One epoch takes 56.43059992790222
Test Error:
 Accuracy: 91.2%, Avg loss: 0.267980

Epoch 20
-------------------------------
loss: 0.012503 [   32/60000]
loss: 0.011884 [ 3232/60000]
```

```
loss: 0.137949 [ 6432/60000]
loss: 0.044050 [ 9632/60000]
loss: 0.047998 [12832/60000]
loss: 0.142378 [16032/60000]
loss: 0.159879 [19232/60000]
loss: 0.259280 [22432/60000]
loss: 0.066730 [25632/60000]
loss: 0.135672 [28832/60000]
loss: 0.133326 [32032/60000]
loss: 0.165349 [35232/60000]
loss: 0.115016 [38432/60000]
loss: 0.210521 [41632/60000]
loss: 0.045307 [44832/60000]
loss: 0.139606 [48032/60000]
loss: 0.097269 [51232/60000]
loss: 0.127427 [54432/60000]
loss: 0.065583 [57632/60000]
One epoch takes 57.63493204116821
Test Error:
 Accuracy: 90.5%, Avg loss: 0.278169

Test Error:
 Accuracy: 90.5%, Avg loss: 0.278169

Training done!
```